

Attention based Encoder Decoder Calculations

CMR@BRACU

Encoder

I go to school Start with embeddings of I, go, to ,school

Add positional encoding to determine the order of the words

Form Query, Key and Value matrix for encoder

Transform each embedding into K, Q and V embedding by multiplying with Query, Key and value matrix

Calculate self attention of I, between I and go, I and to, I and school and update the V embedding of I. Now calculate the self attention of go, between go and I, go and to, go and school and update the value embedding of go. Do the same for to and school

Add the V embedding of each token to their original embedding

Decoder

<EOS>

Form a new set of Query, Key and Value matrix for decoder and transform <EOS> embedding into Q embedding, K embedding and V embedding

Calculate self attention of EOS, update the V embedding of EOS and add the V embedding to the original embedding of EOS

Form a new set of Query, Key and Value matrix for cross attention between encoder and Decoder and transform the encoding to I, go, to, school and EOS

Calculate the attention between EOS and I, EOS and go, EOS and to and EOS and school. Update the V embedding of EOS. Add this V embedding of EOS to its latest embedding

Feed latest embedding of EOS to a dense network to produce the embedding of আমি

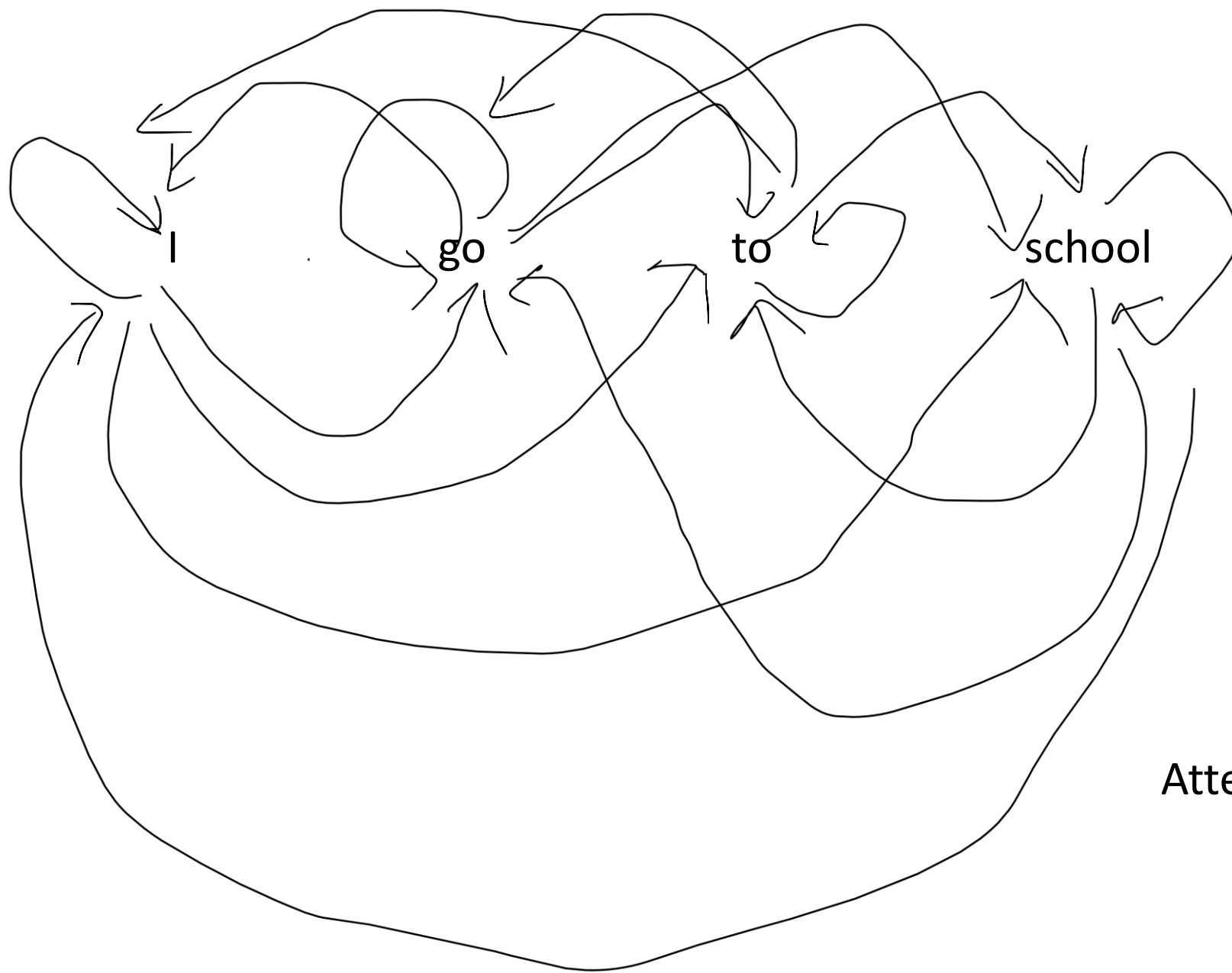
Embedding of আমি is converted to Q, K and V embedding using matrices for decoder

Calculate self attention of আমি and between আমি and <EOS>. Update the V embedding of আমি and add this V embedding with its original embedding

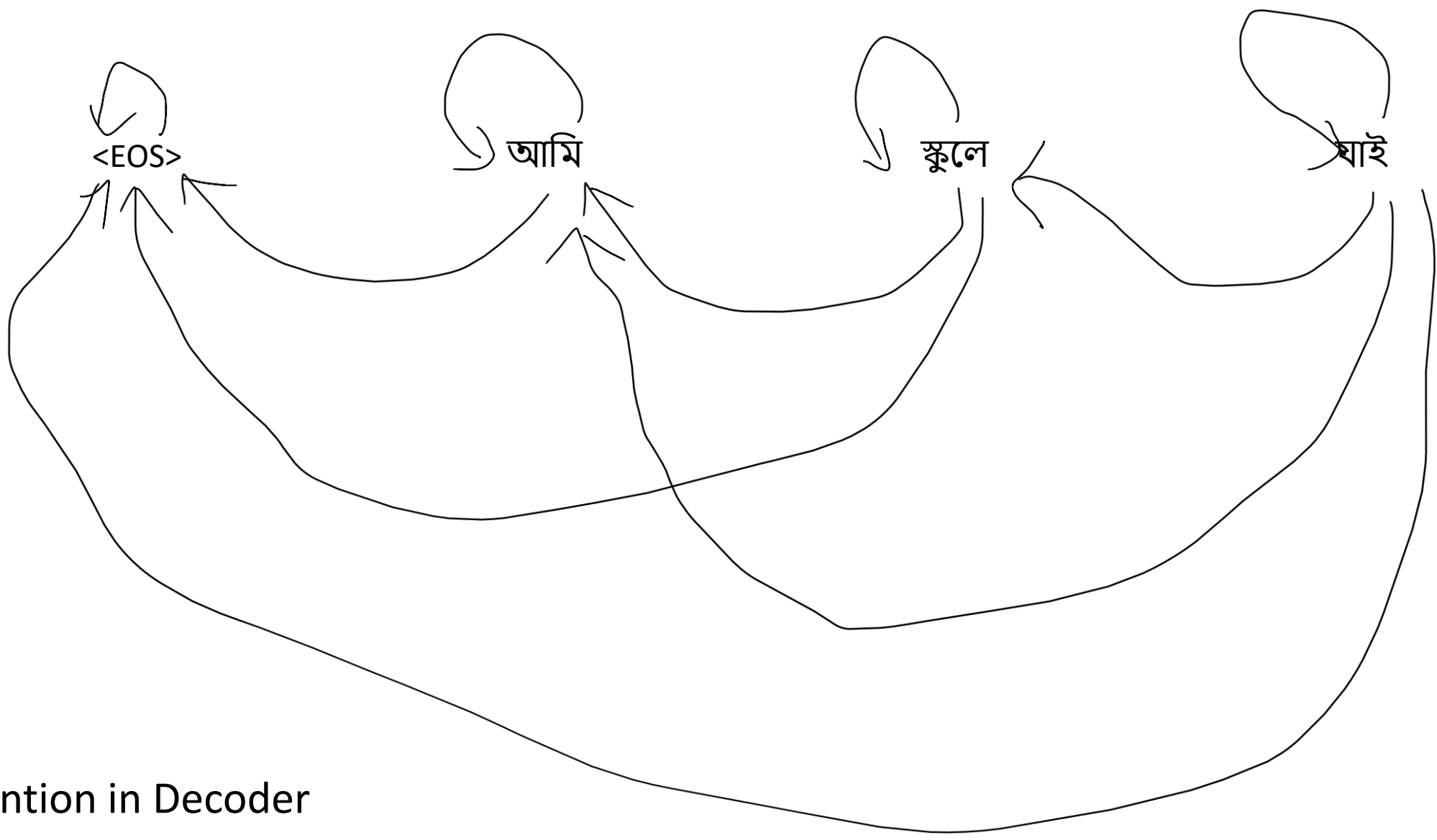
Now convert the latest embedding of আমি to Q, K and V embedding using the matrices of cross attention

Calculate the attention between আমি and I, আমি and go, আমি and to and আমি and School. Update the V encoding of আমি and add this with the latest encoding of আমি

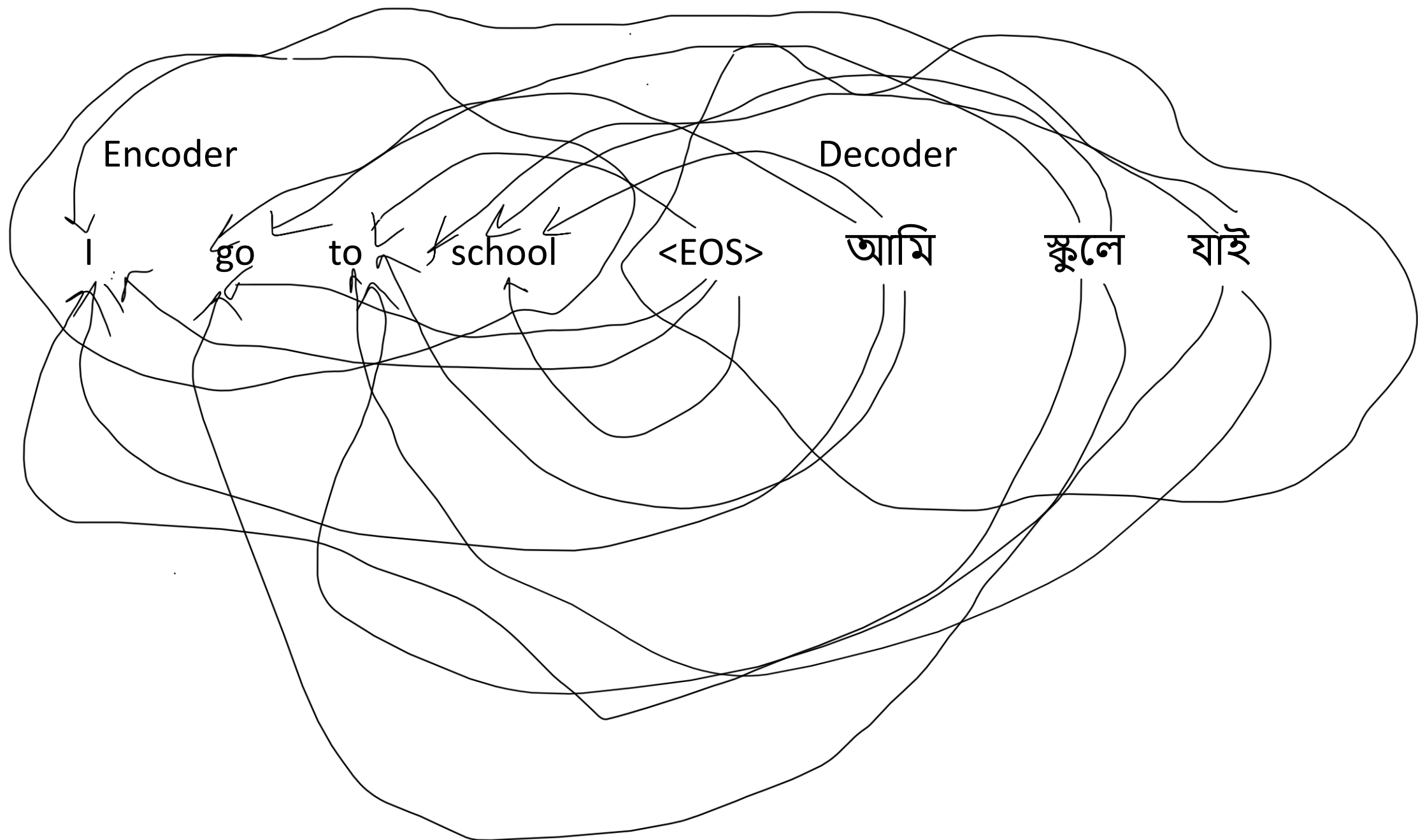
Feed the latest encoding of আমি to a dense neural network and produce the second word Embedding for স্কুলে



Attention in Encoder



Attention in Decoder



Decoder encoder cross attention

Embedding of স্কুলে is converted to Q, K and V embedding using matrices for decoder

Calculate self attention of স্কুলে and between স্কুলে and আমি and স্কুলে and <EOS>.

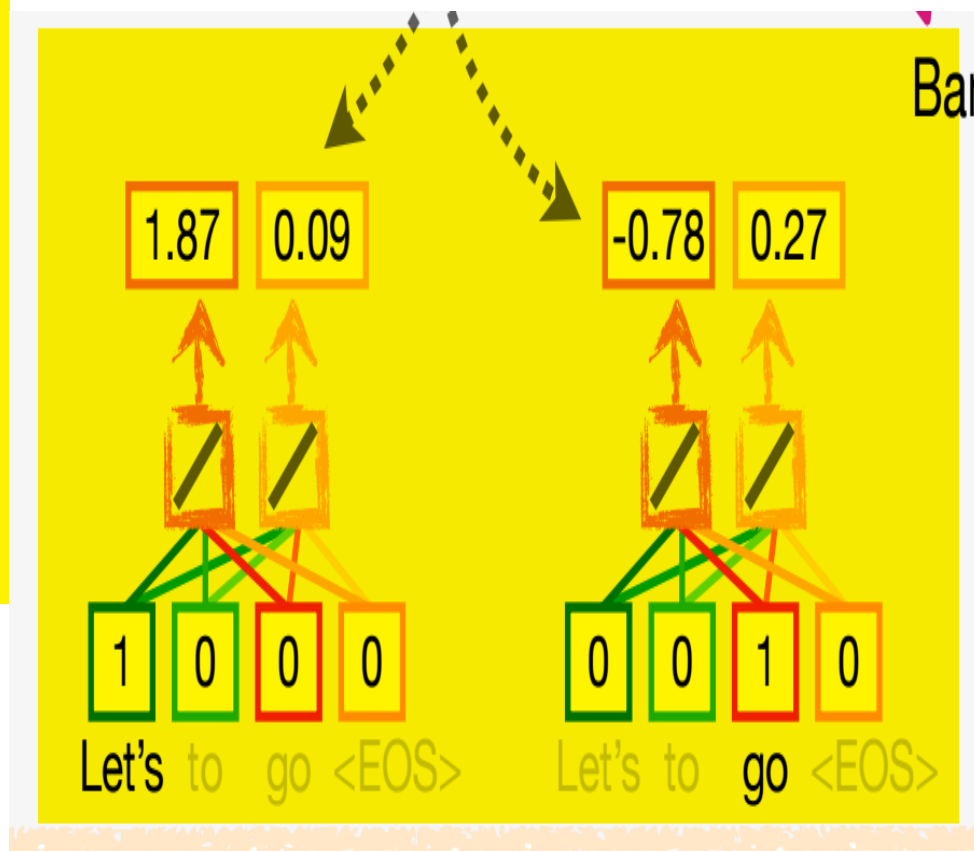
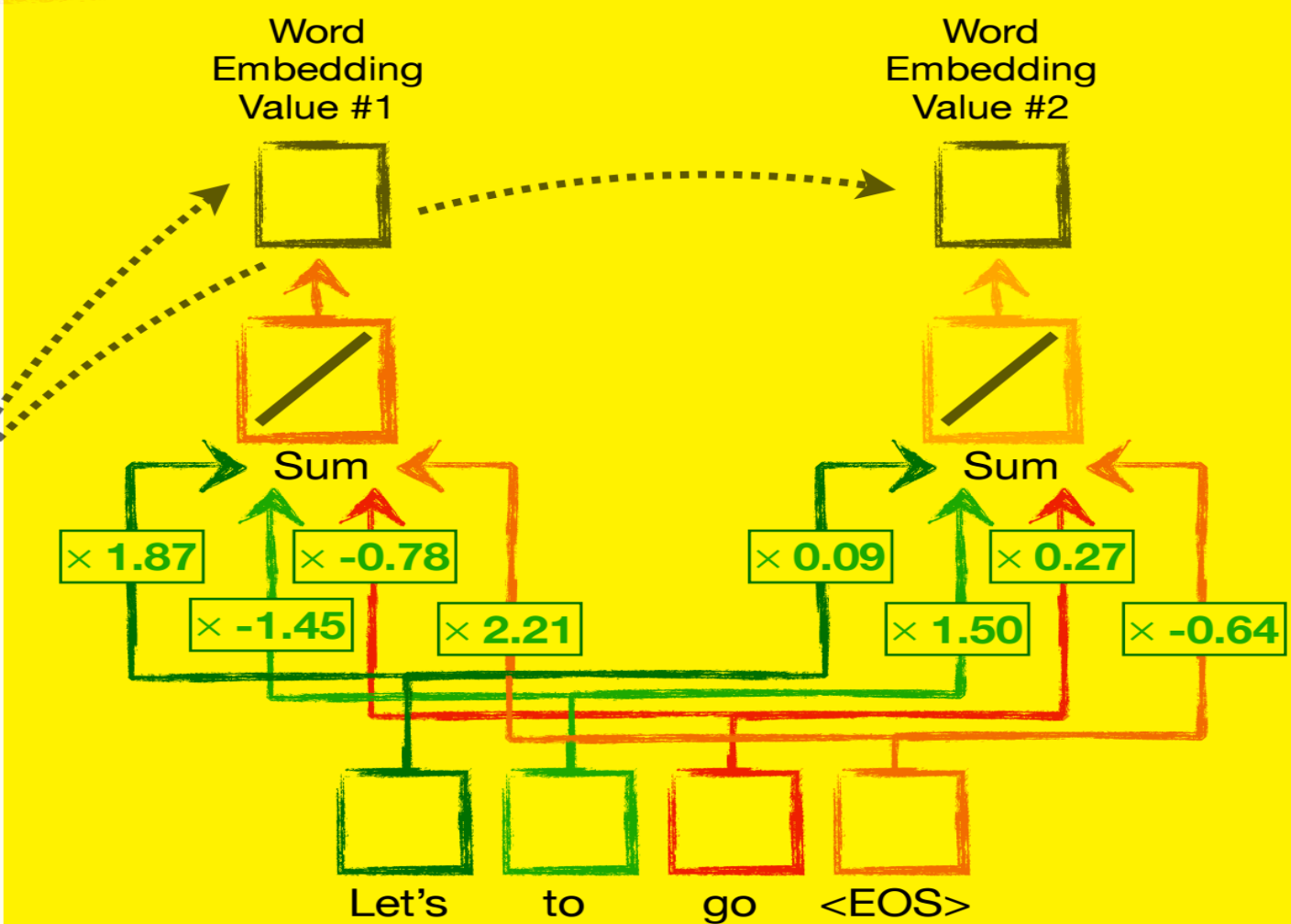
Update the V embedding of স্কুলে and add this V embedding with its original embedding

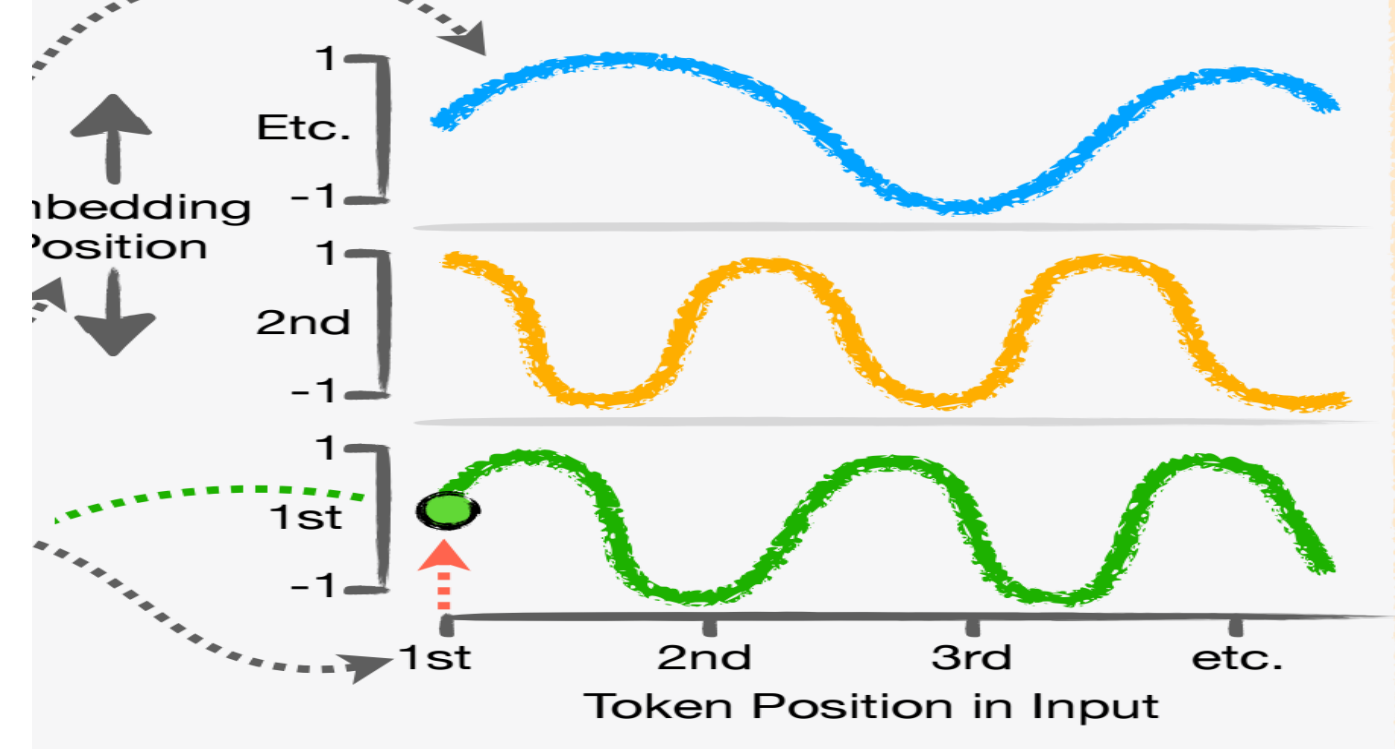
Now convert the latest embedding of স্কুলে to Q, K and V embedding using the matrices of cross attention

Calculate the attention between স্কুলে and I, স্কুলে and go, স্কুলে and to and স্কুলে and School. Update the V encoding of স্কুলে and add this with the latest encoding of স্কুলে

Feed the latest encoding of স্কুলে to a dense neural network and produce the third word
Embedding for যাই

Update the embedding of যাই as done for other tokens of decoder and lastly produce <EOS>





$$PE(pos, 2i) = \sin \frac{pos}{10000^{\frac{2i}{d_{model}}}}$$

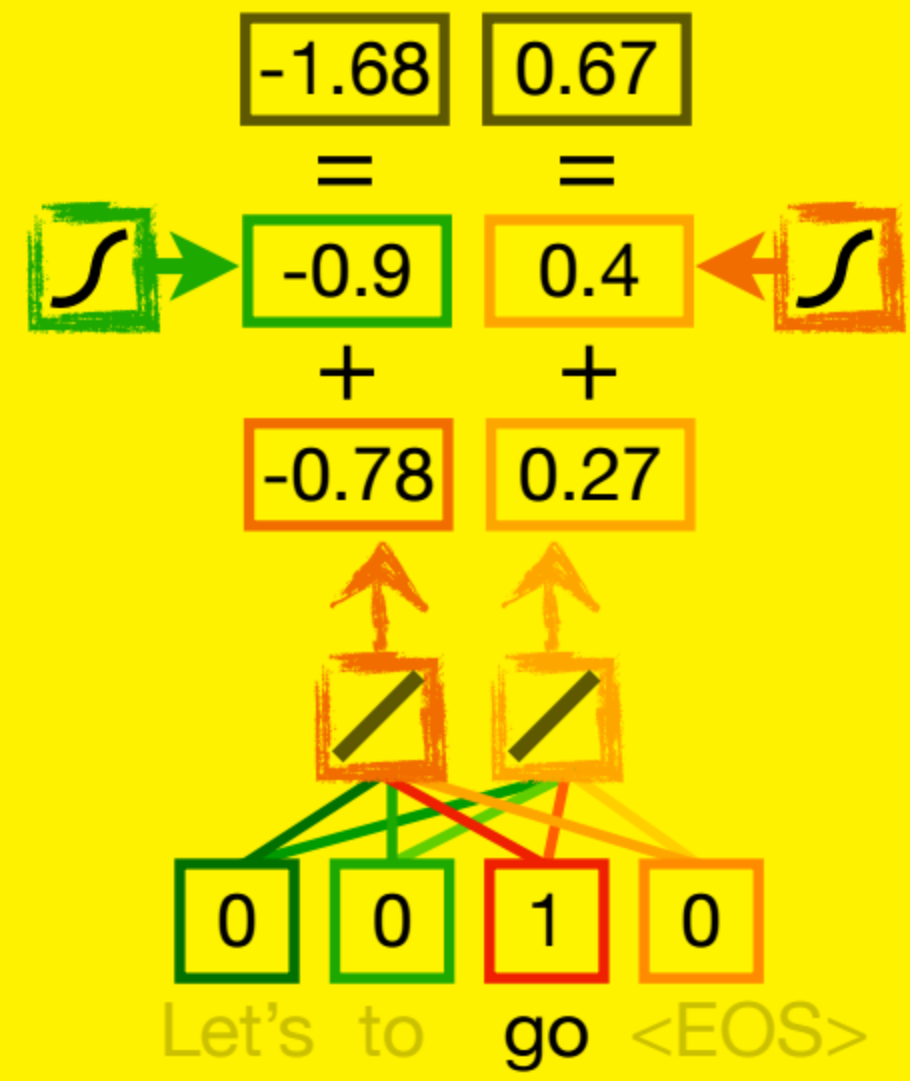
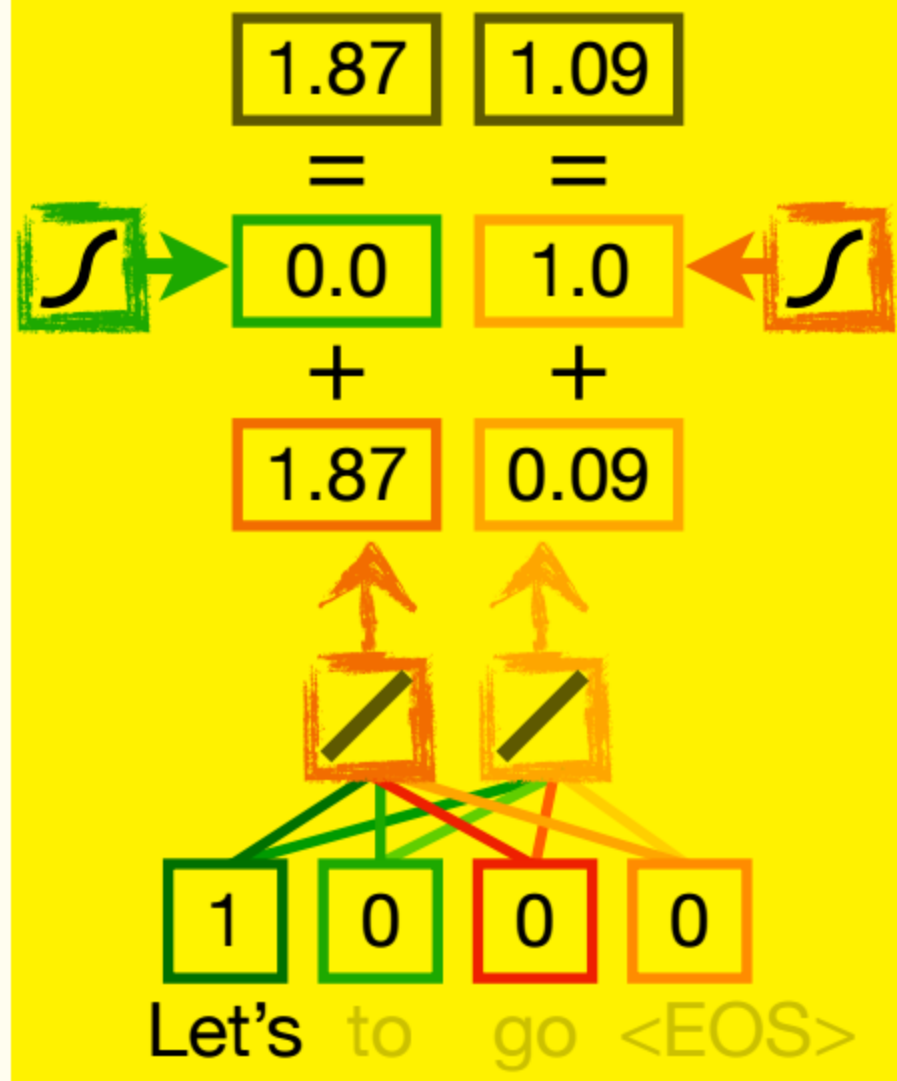
$$PE(pos, 2i+1) = \cos \frac{pos}{10000^{\frac{2i}{d_{model}}}}$$

$$PE(0,0) = \sin(0) = 0, PE(0,1) = \cos(0), PE(0,2) = \sin(0), PE(0,3) = 1$$

$$PE(1,0) = \sin(1), PE(1,1) = \cos(1), PE(1,2) = \sin(1/10000^{(2/4)}), PE(1,3) = \cos(1/10000^{(2/4)})$$

word1	word2	word3
.34 .56 .78 .90	.56 .64 .88 .76	.34 .12 .39 .98

.34+0, .56+1, .78+0, .90+1	.56+0.02, .64+1, .88+0, .76+1
----------------------------	-------------------------------



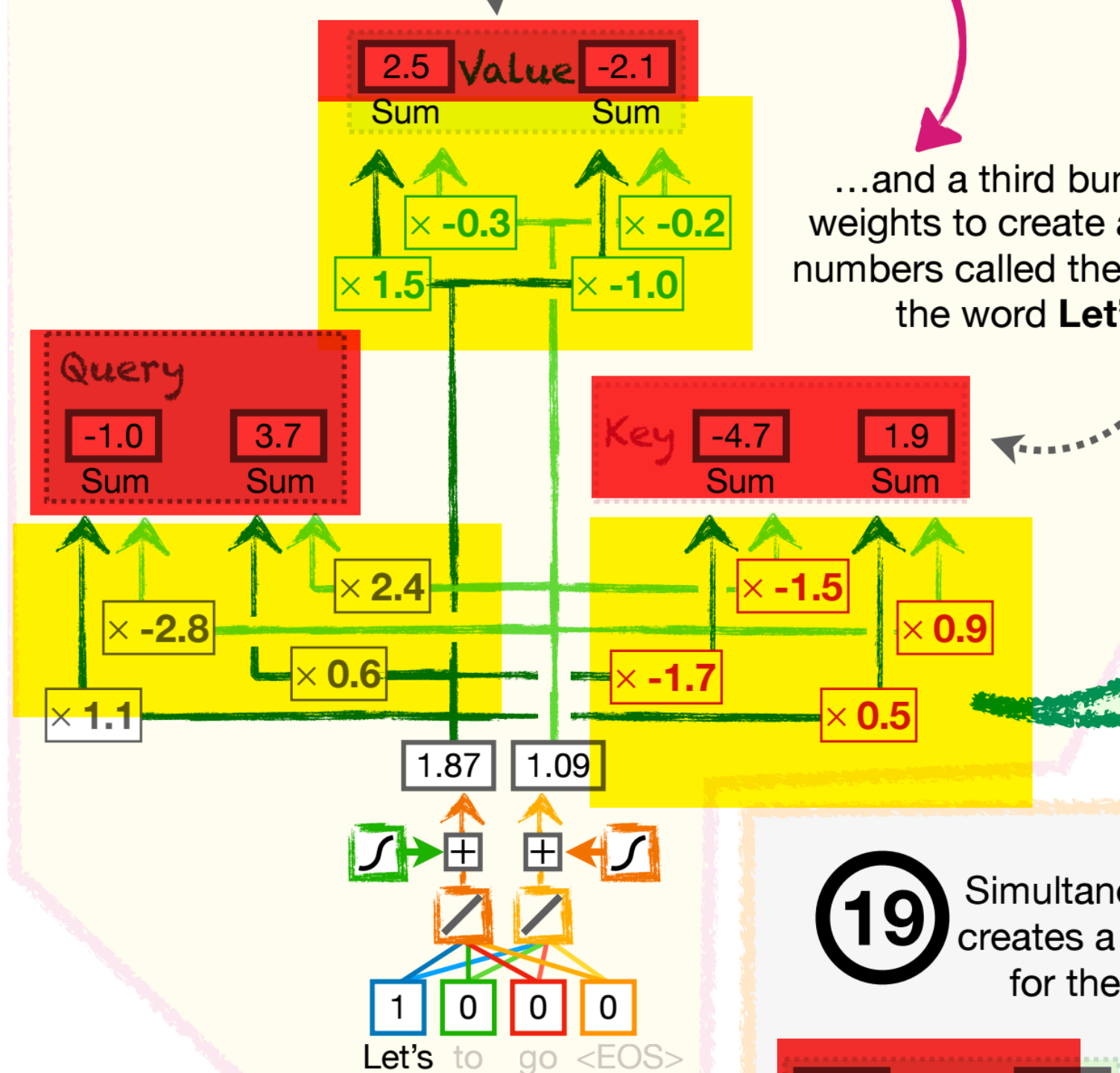


The **pizza** came out of the **oven**, and **it** tasted good!

This diagram illustrates the Self-Attention mechanism in a Transformer model. It features two identical sentences: "The **pizza** came out of the **oven**, and **it** tasted good!". The top sentence is the input, and the bottom sentence is the output. The word "good" in the top sentence is highlighted in red. Multiple green curved arrows originate from the word "good" and point to each word in the bottom sentence, representing the attention weights. A dashed green arrow points from the text "Self-Attention would calculate the Similarity Scores between good and all of the words, including itself." to the word "good" in the top sentence. A dashed green arrow also points from the word "good" in the top sentence to the word "good" in the bottom sentence, indicating the self-attention connection.

...and, ultimately, Self-Attention would calculate the Similarity Scores between **good** and all of the words, including itself.

The **pizza** came out of the **oven**, and **it** tasted good!



19 Simultane
creates a
for the

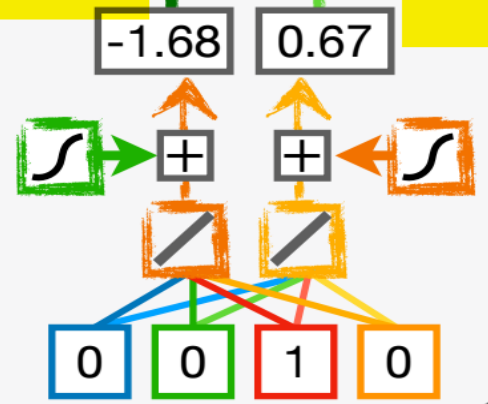
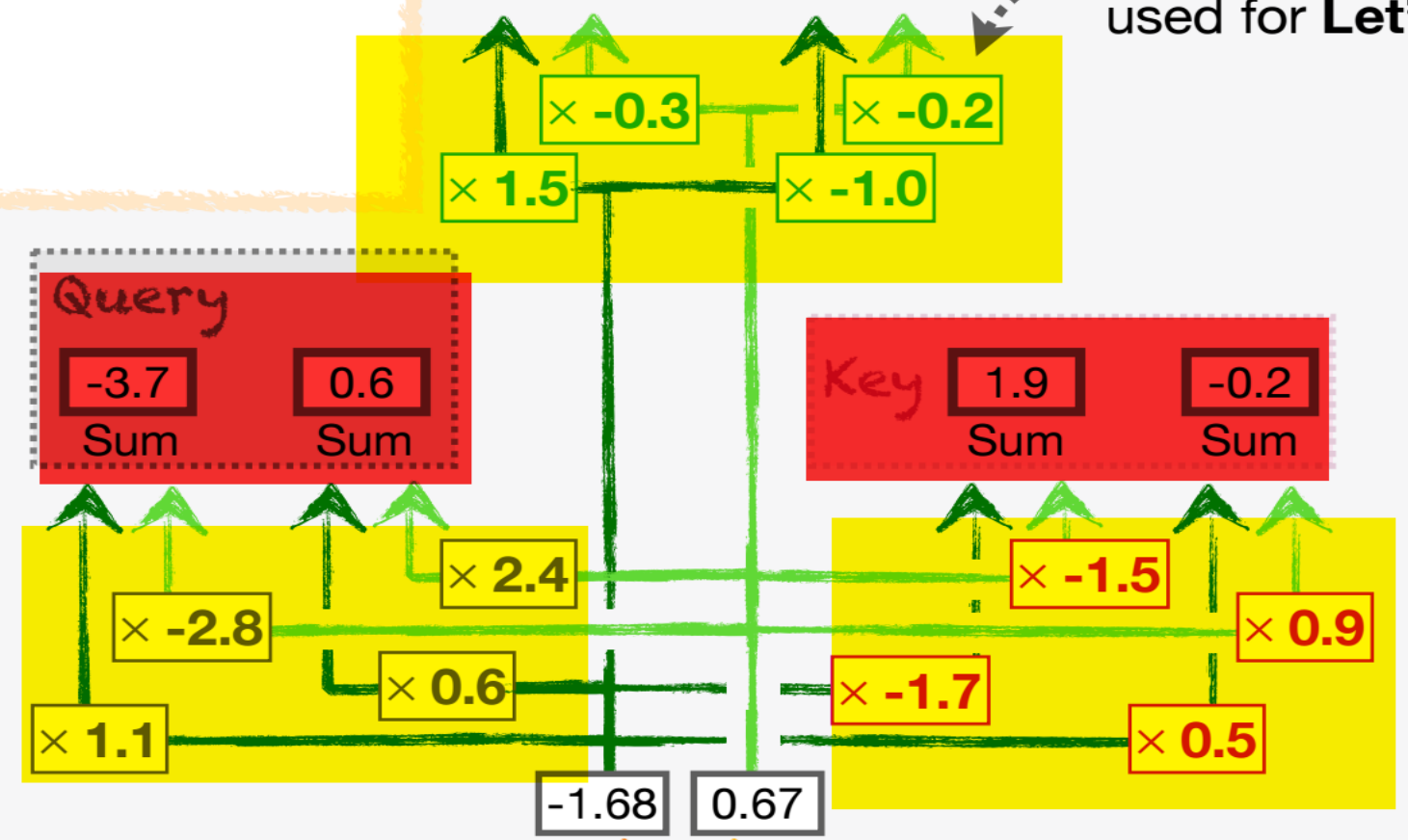
Let's to go <EOS>

Value
Sum: -2.7 Sum: 1.5

...using the exact same sets of weights that we used for **Let's**.

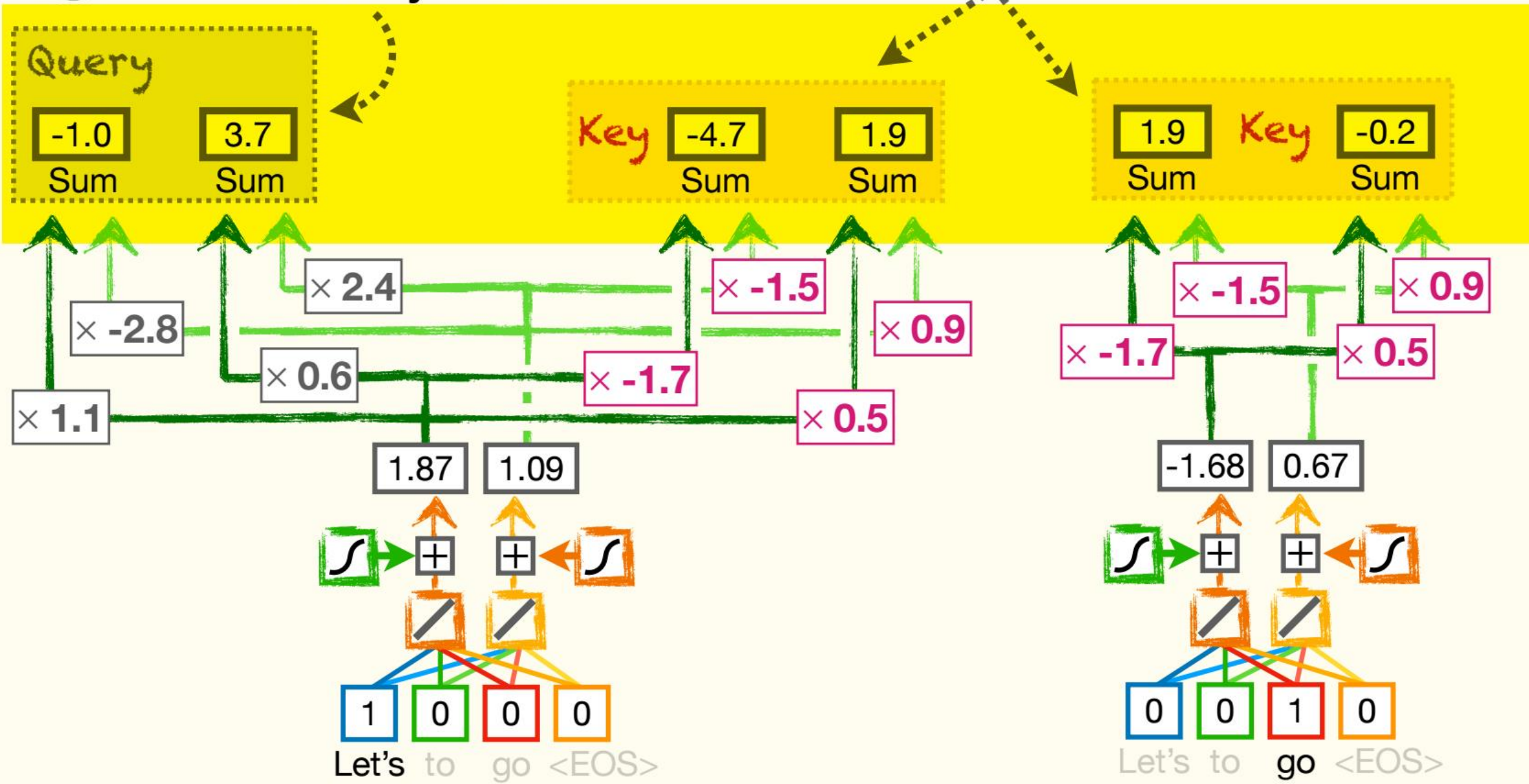
Query
Sum: -3.7 Sum: 0.6

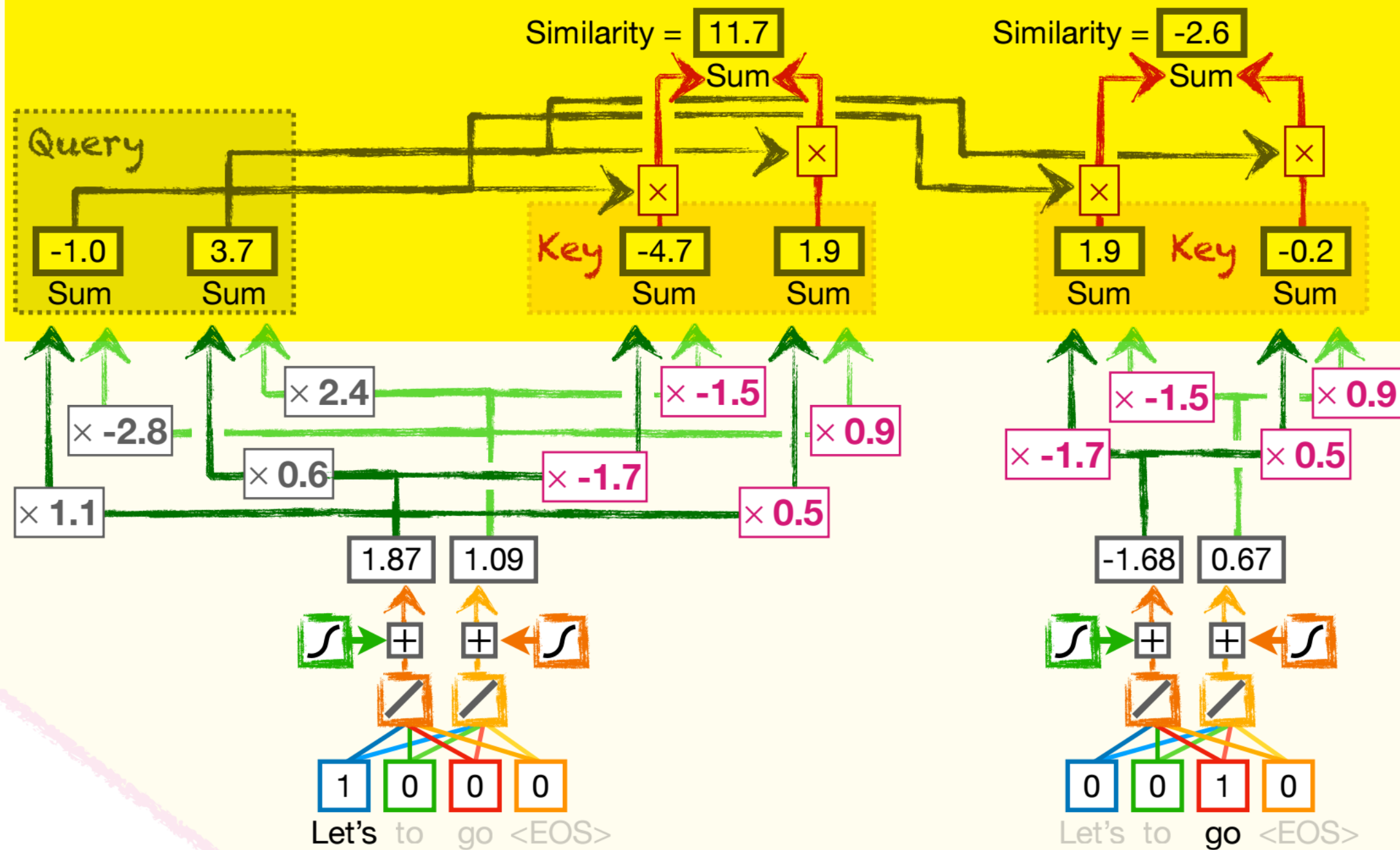
Key
Sum: 1.9 Sum: -0.2

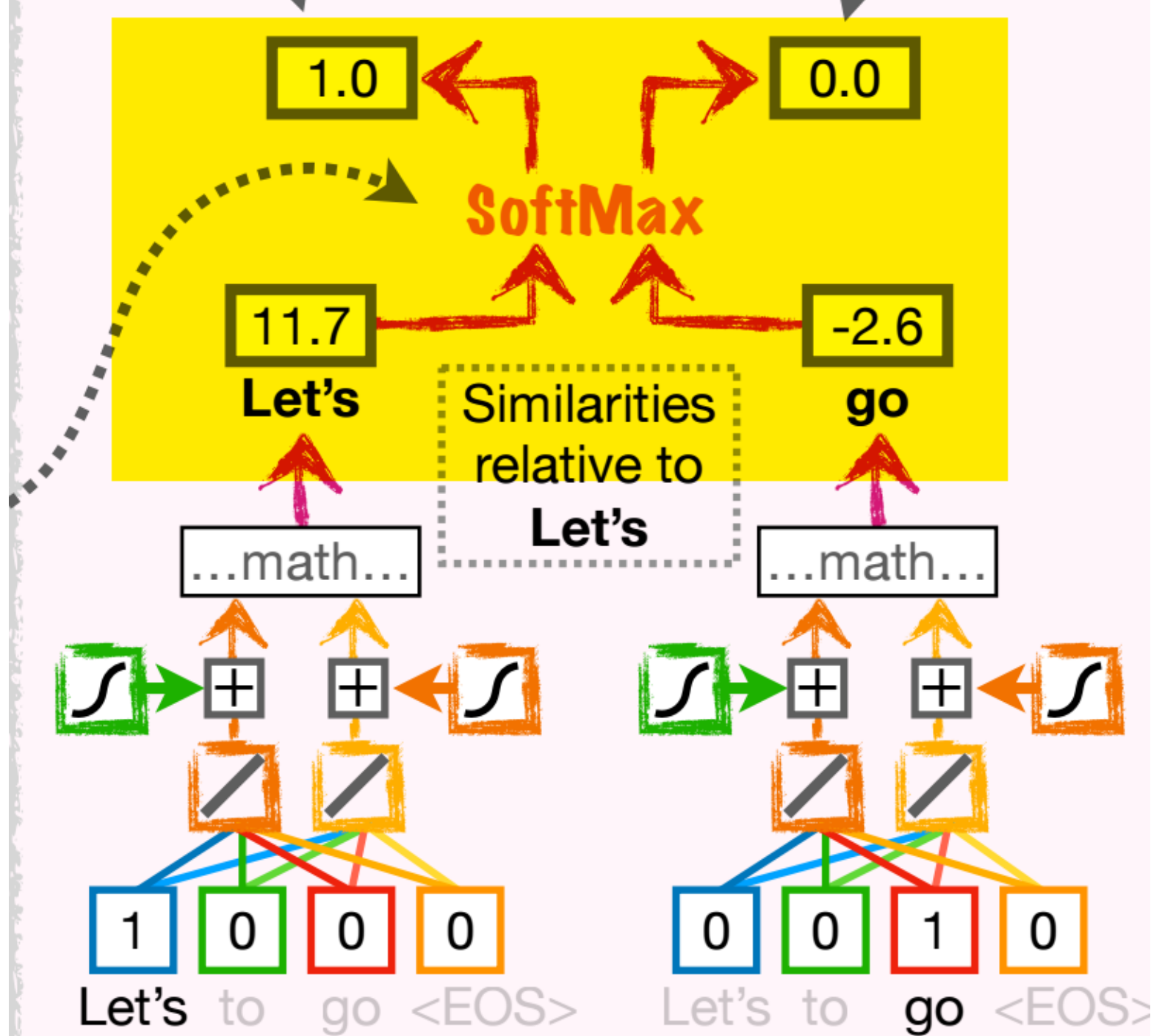


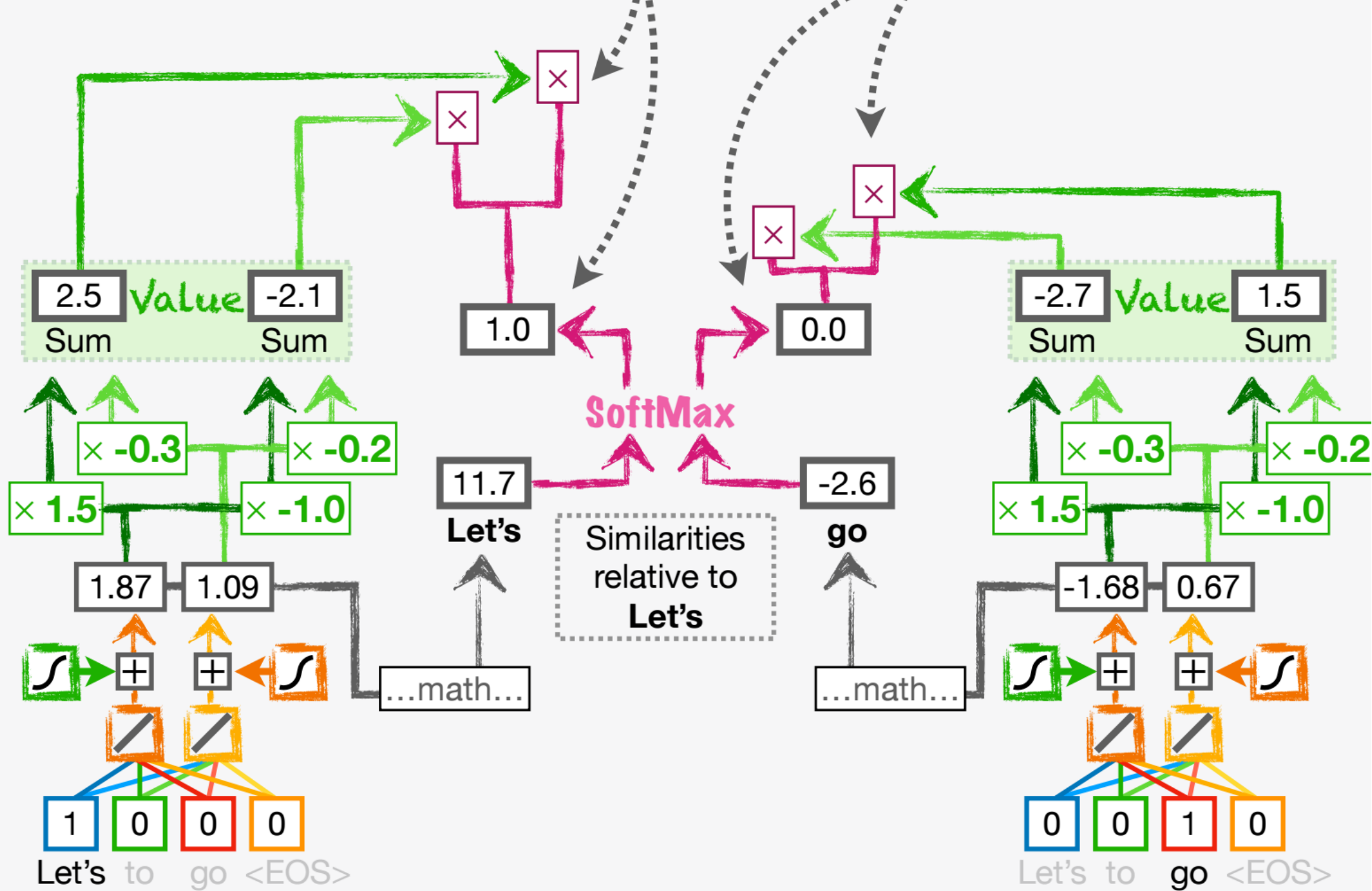
Let's to go <EOS>

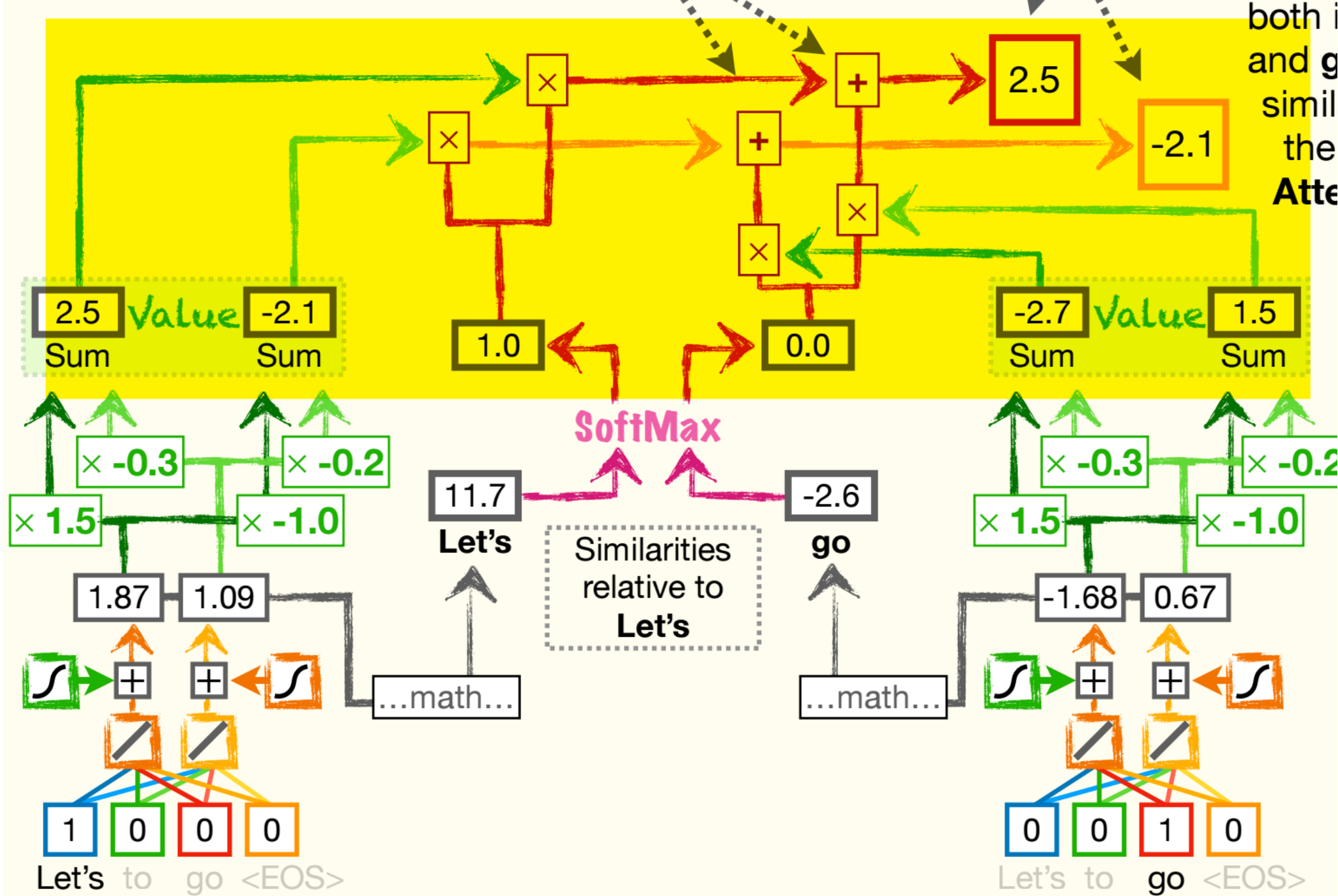
the Query for Let's...

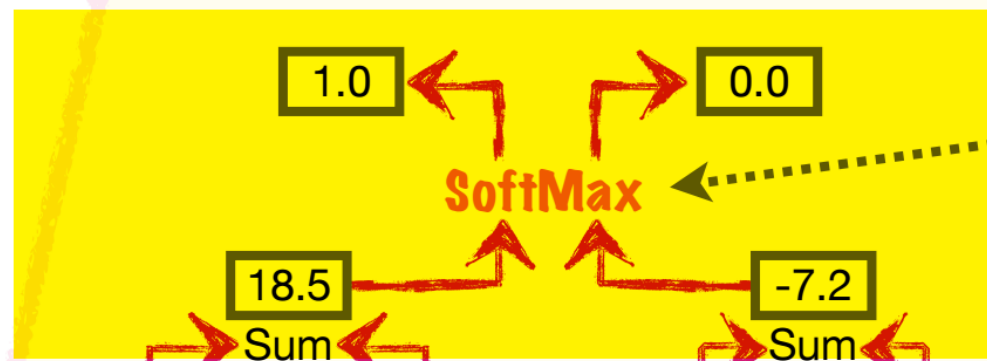




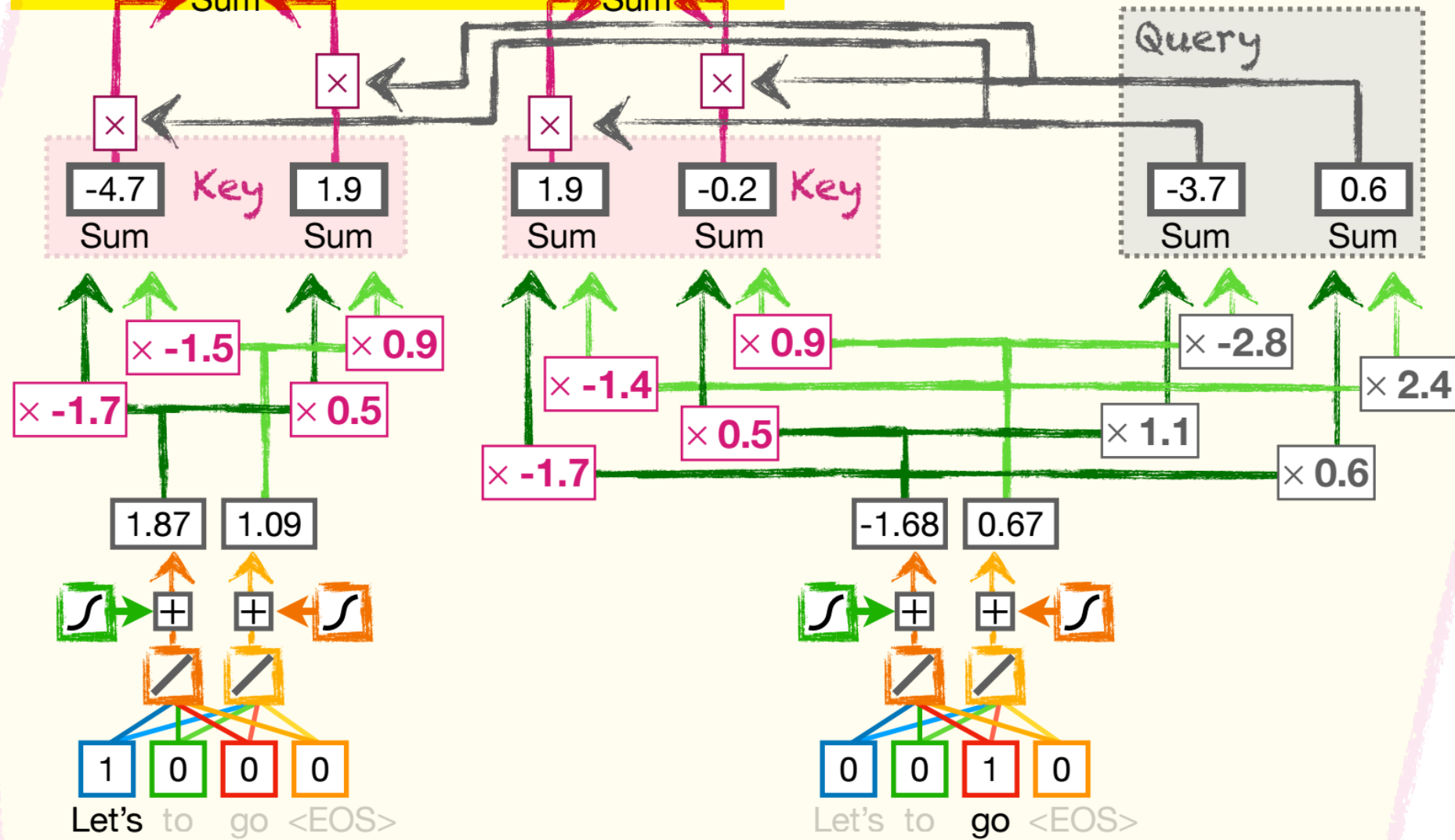




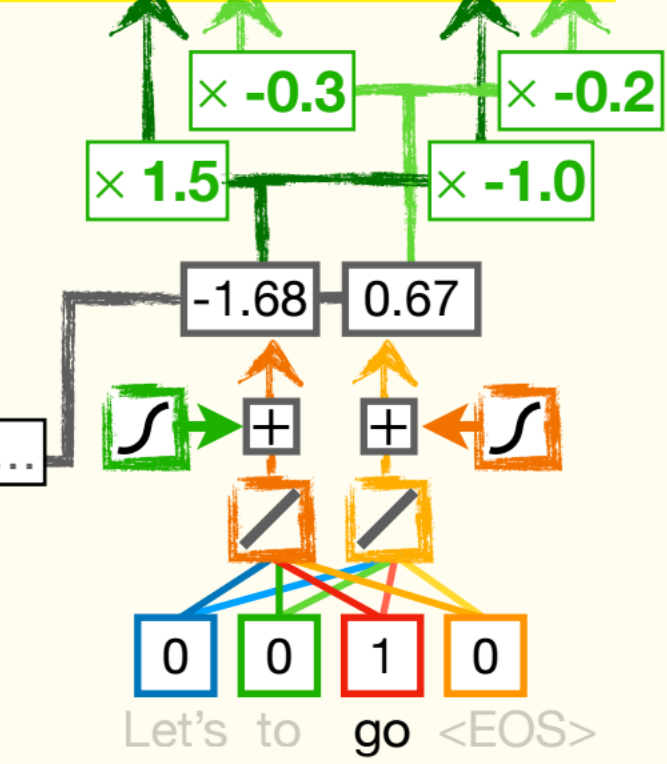
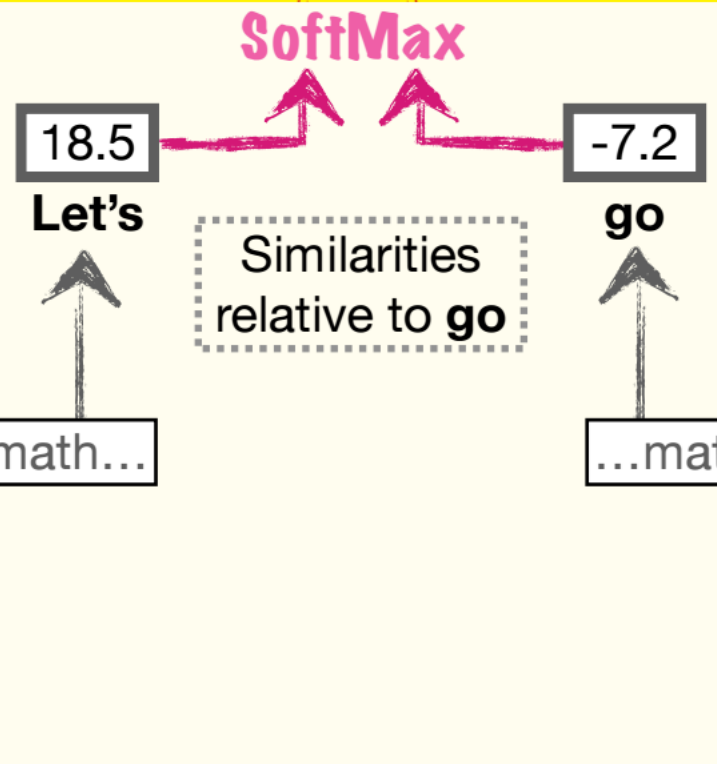
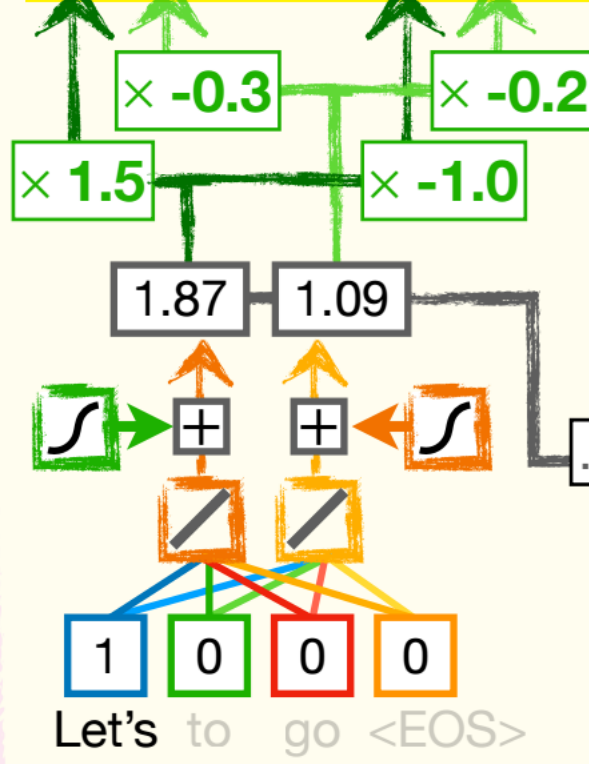
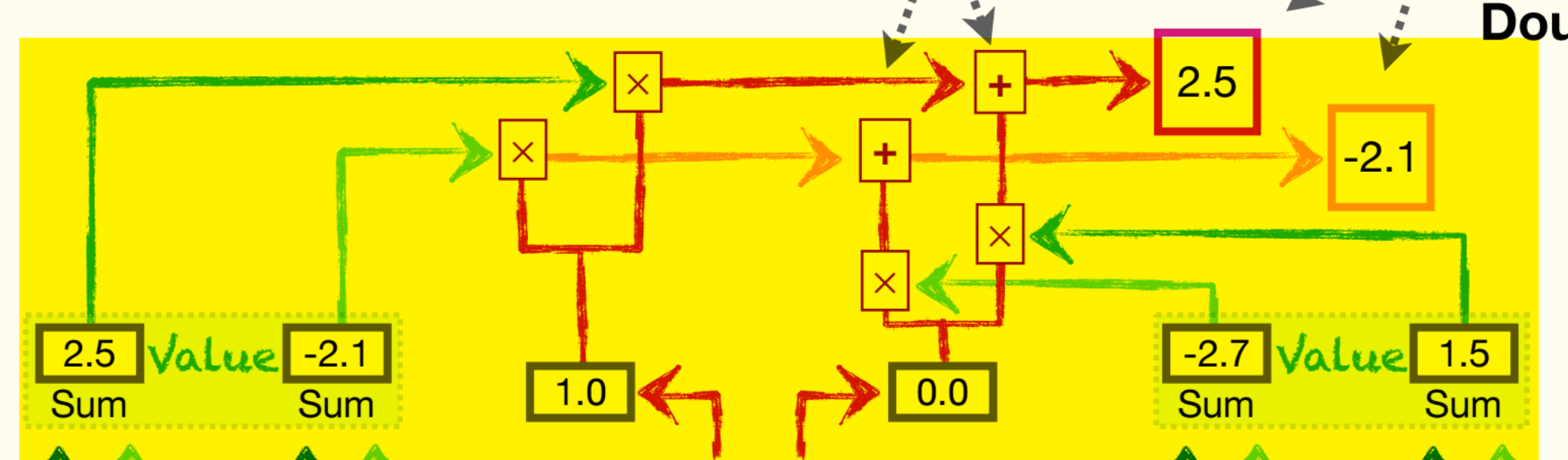


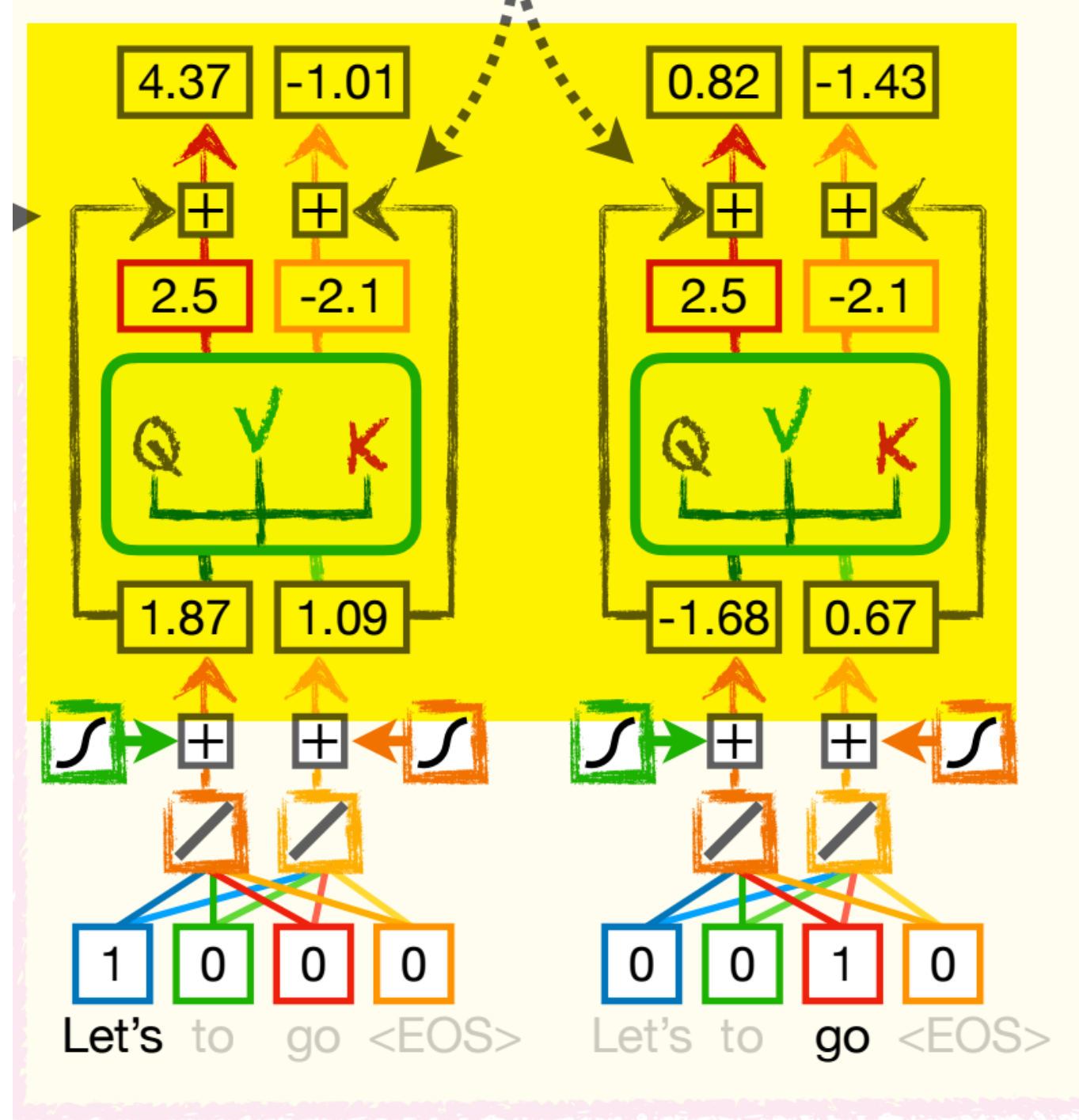


Then we run the similarities through the SoftMax function to determine what percentage of each token v should use to encode **go**...

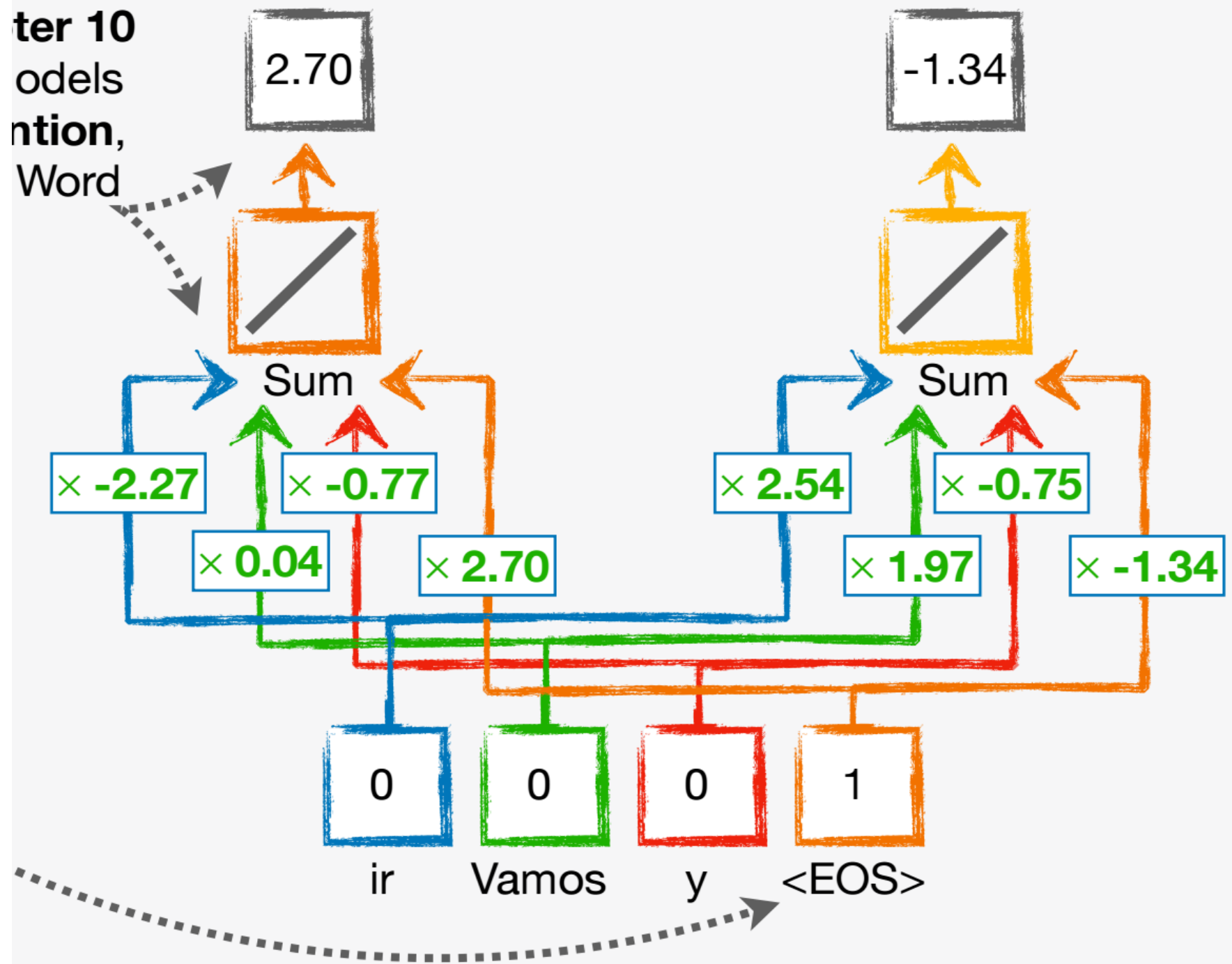


Double



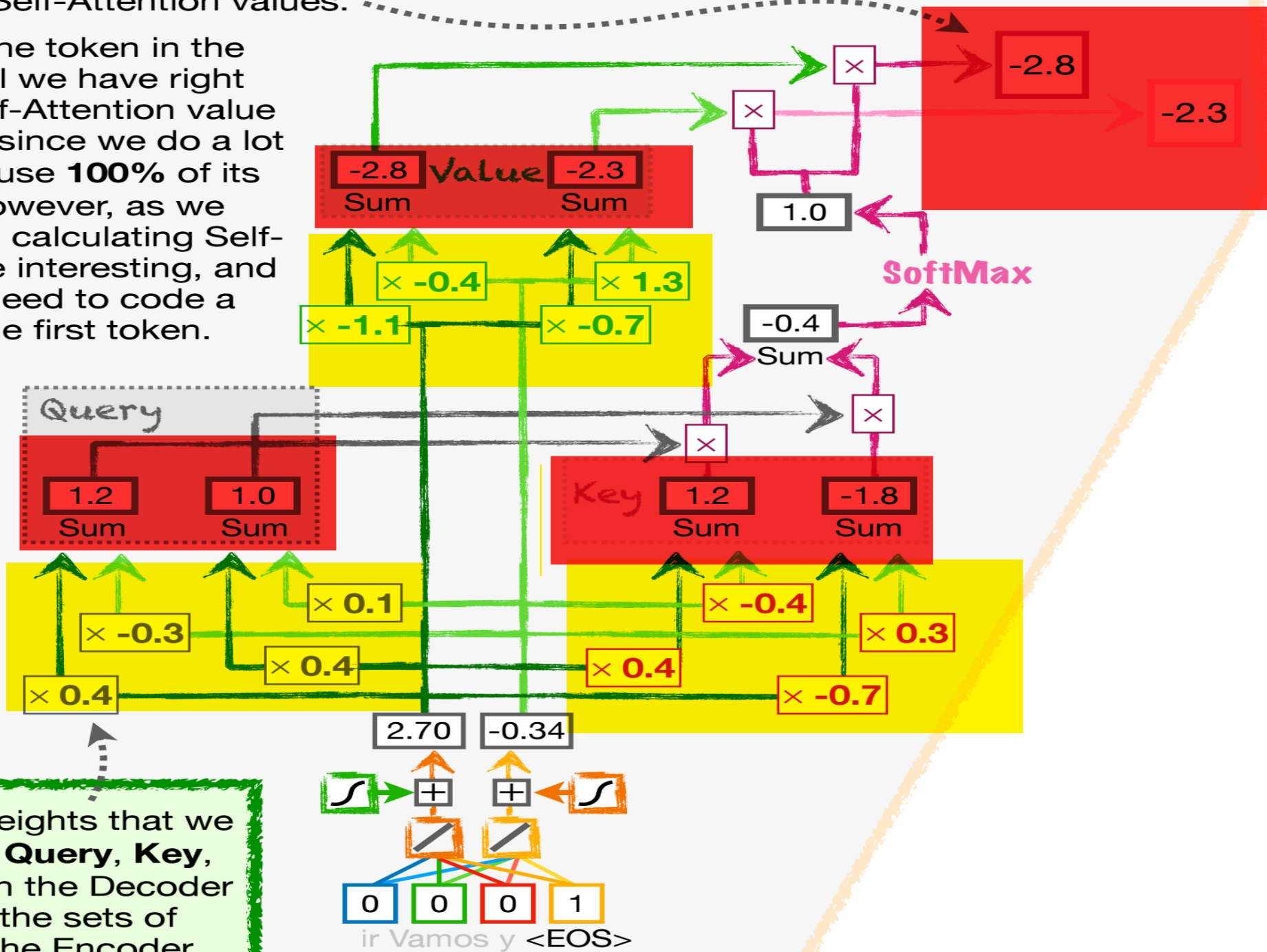


ter 10
odels
ntion,
Word

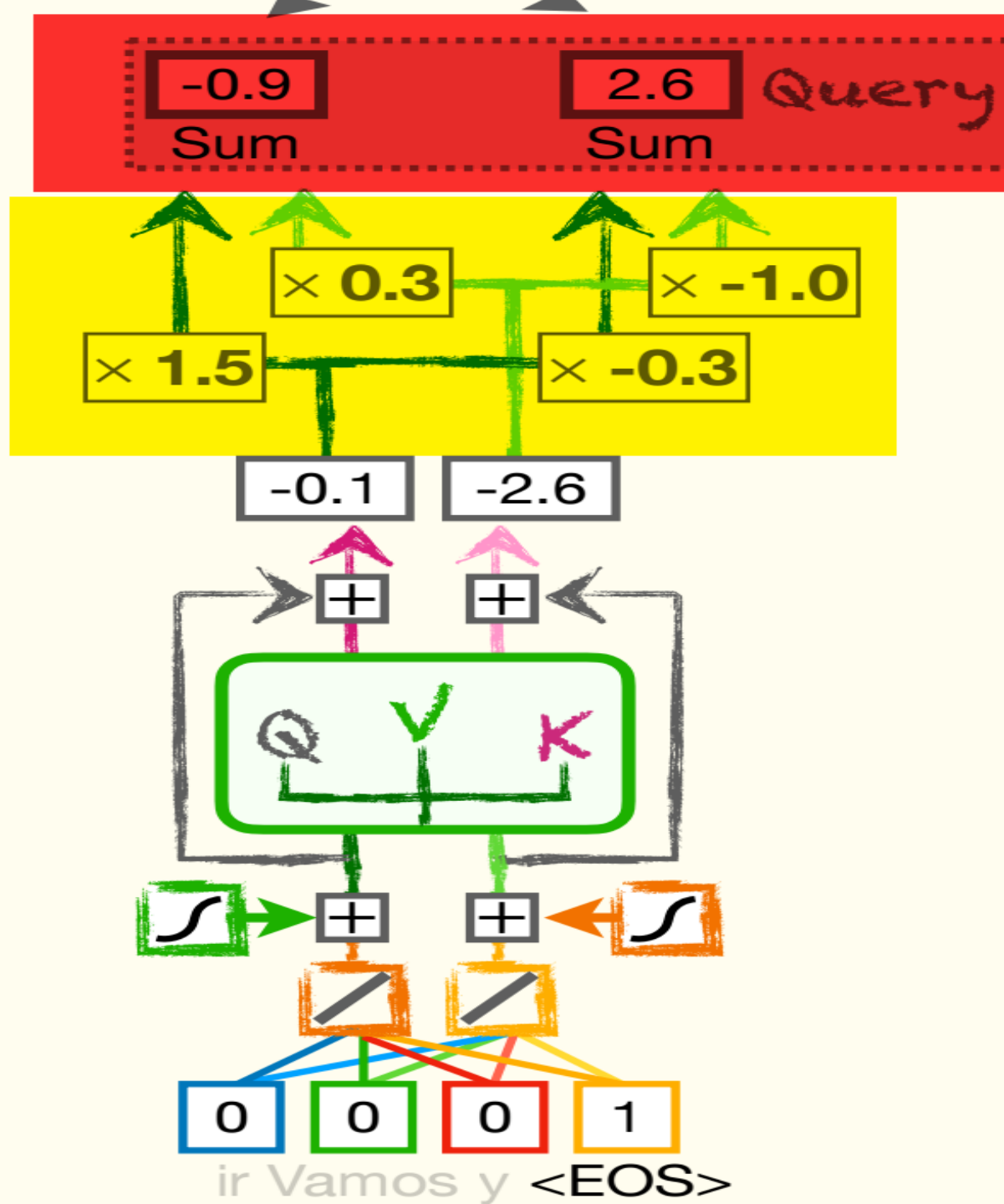


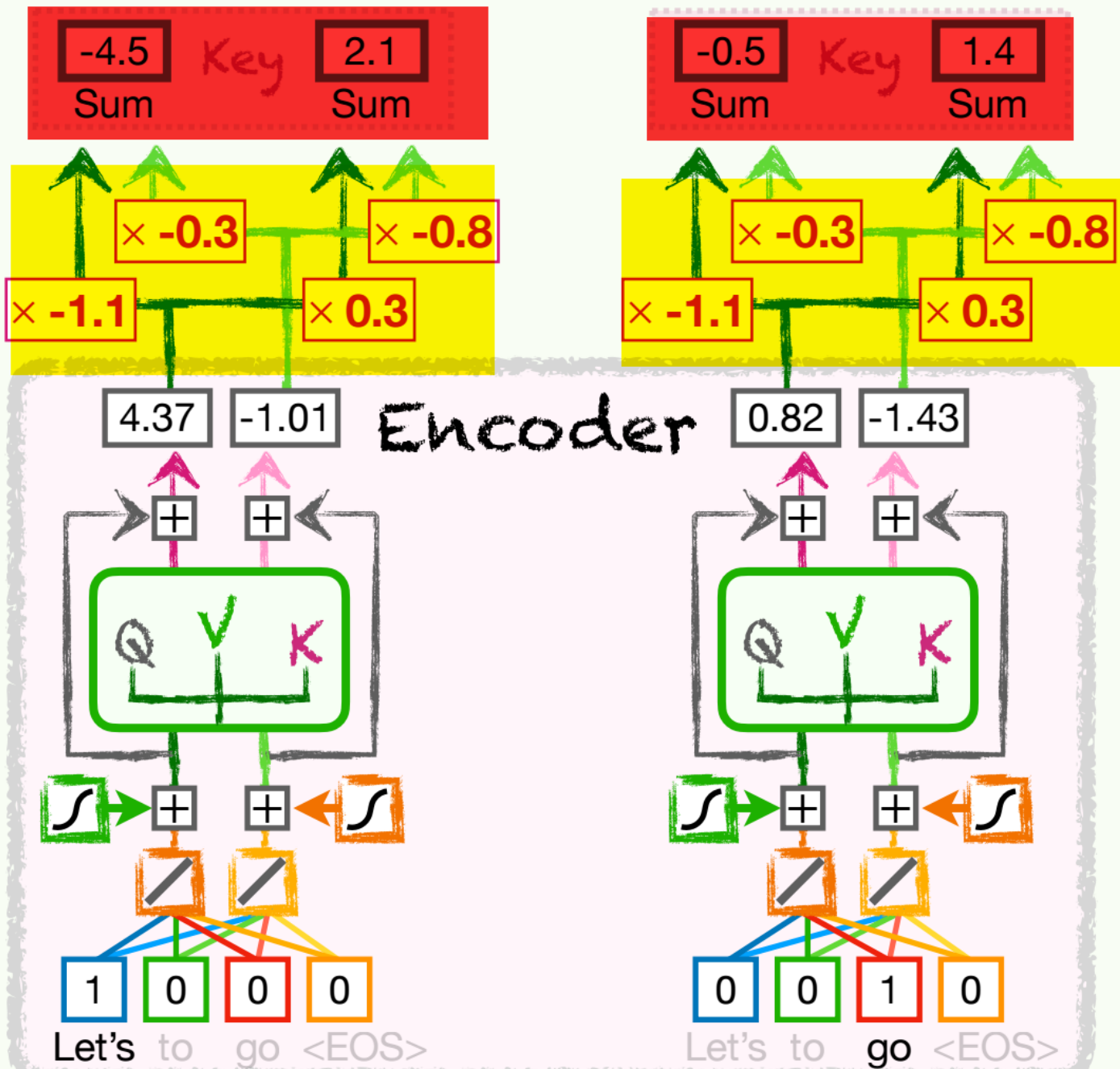
Transformer, we calculate the individual Self-Attention values.

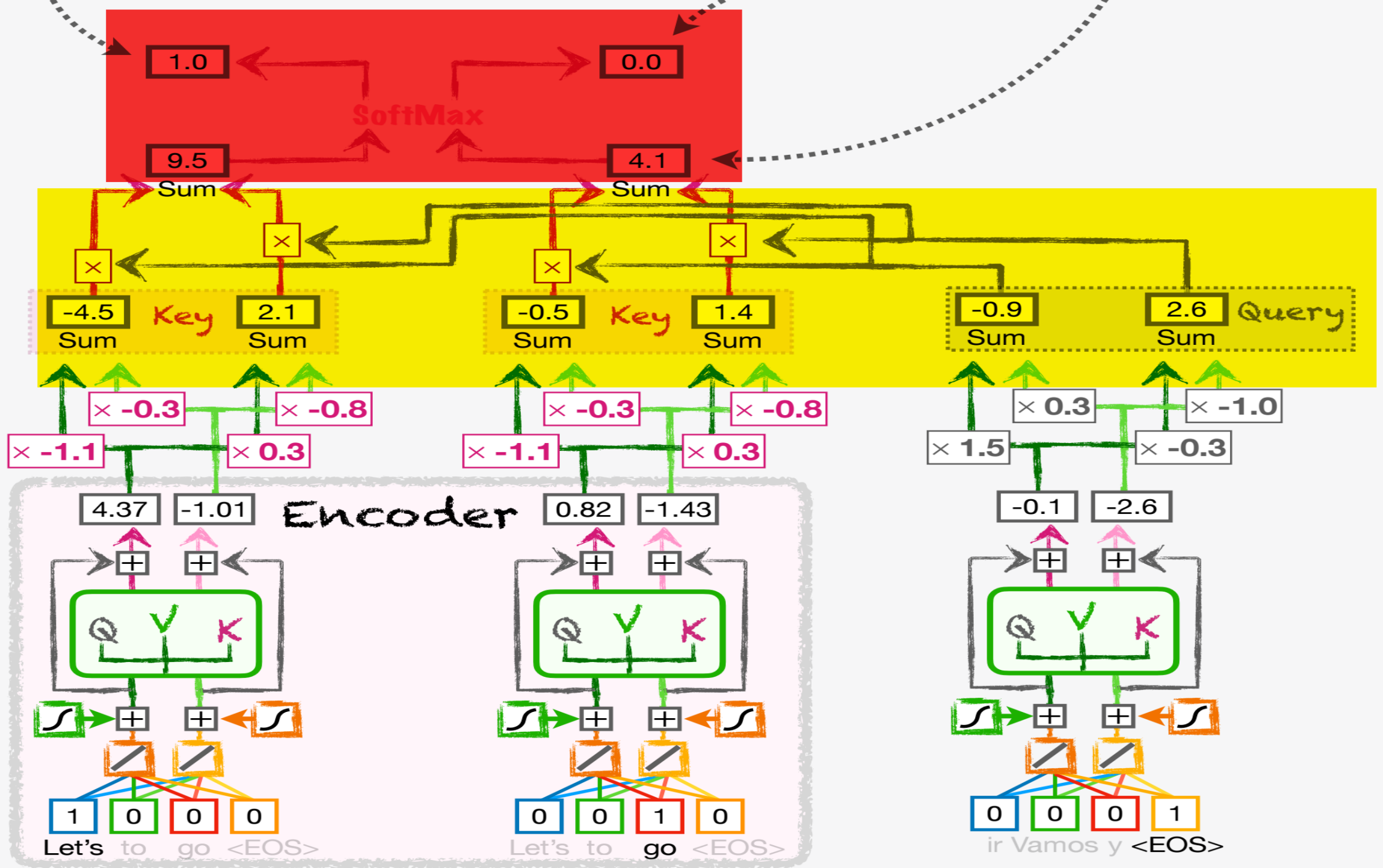
only one token in the
which is all we have right
by a Self-Attention value
is silly, since we do a lot
just to use **100%** of its
resources. However, as we
process tokens, calculating Self-
Attention is more interesting, and
we don't need to code a
loop for the first token.

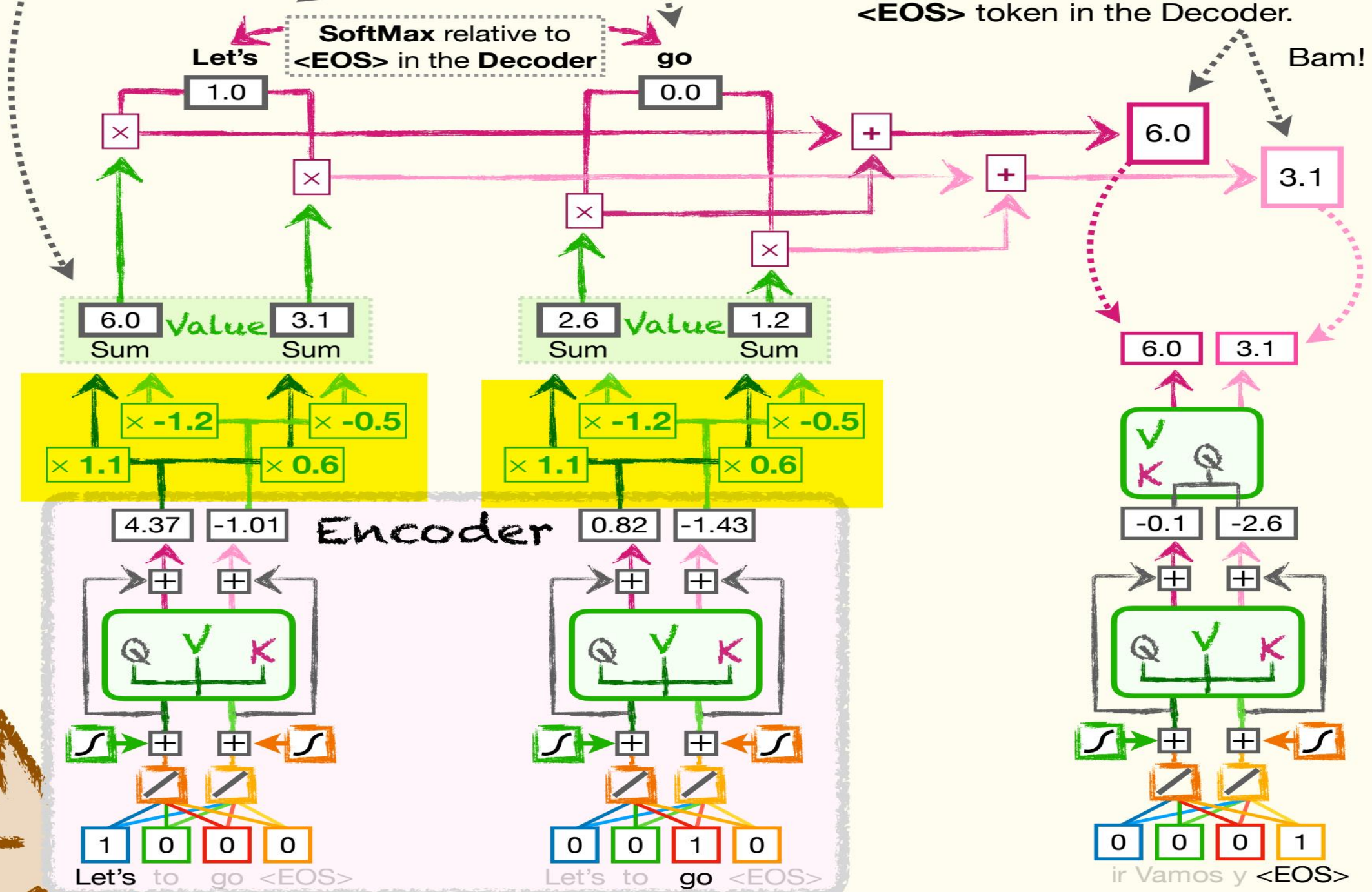


sets of weights that we
use to calculate the **Query**, **Key**,
and **Value** vectors in the Decoder
are learned from the sets of
weights used in the Encoder.



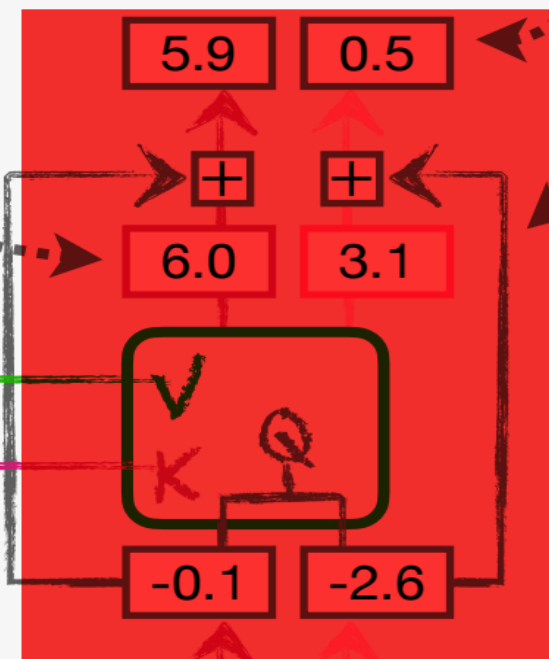
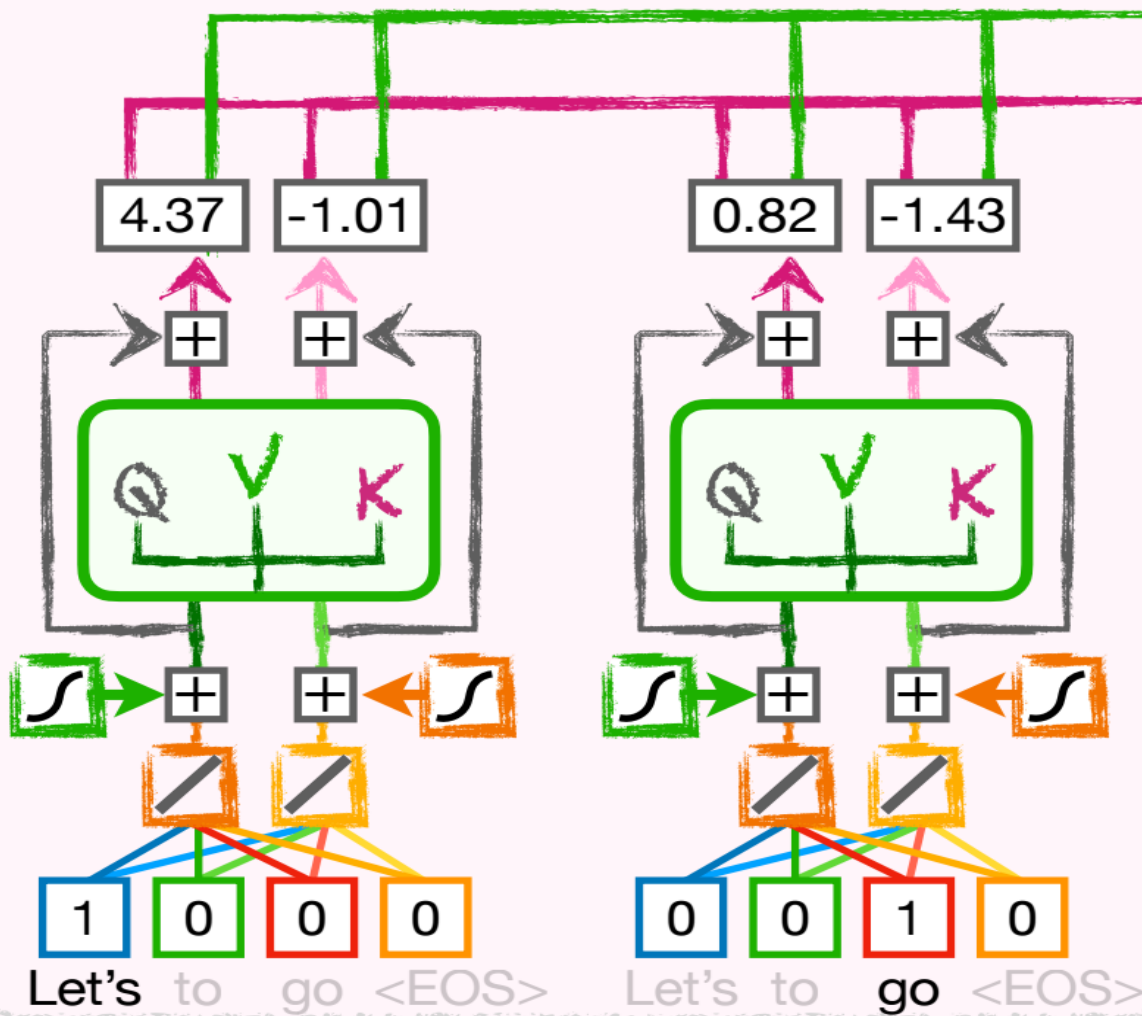




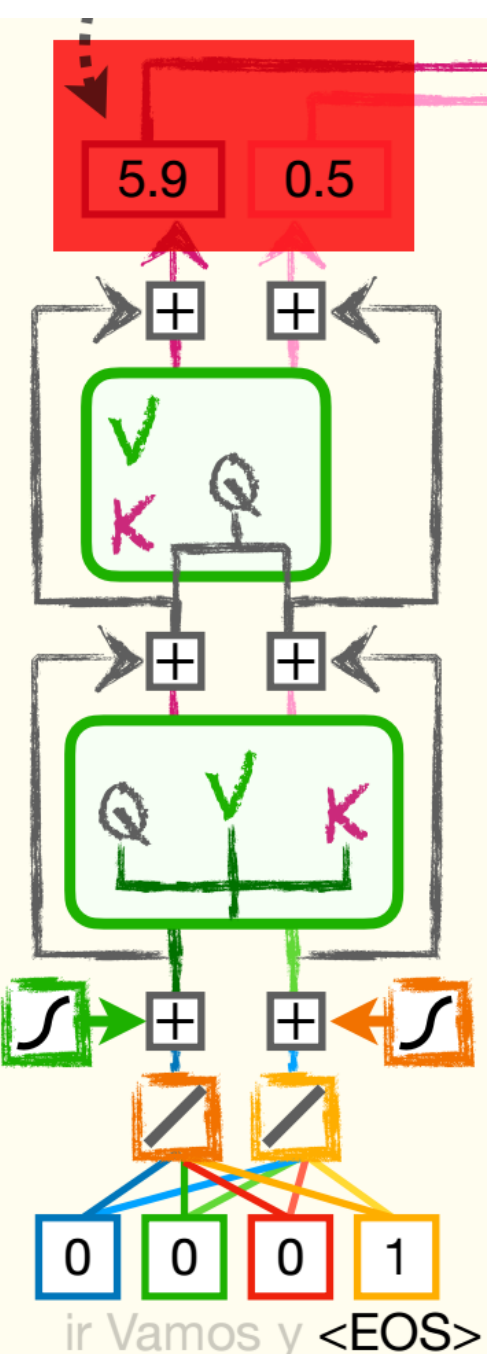


for the <EOS> token...

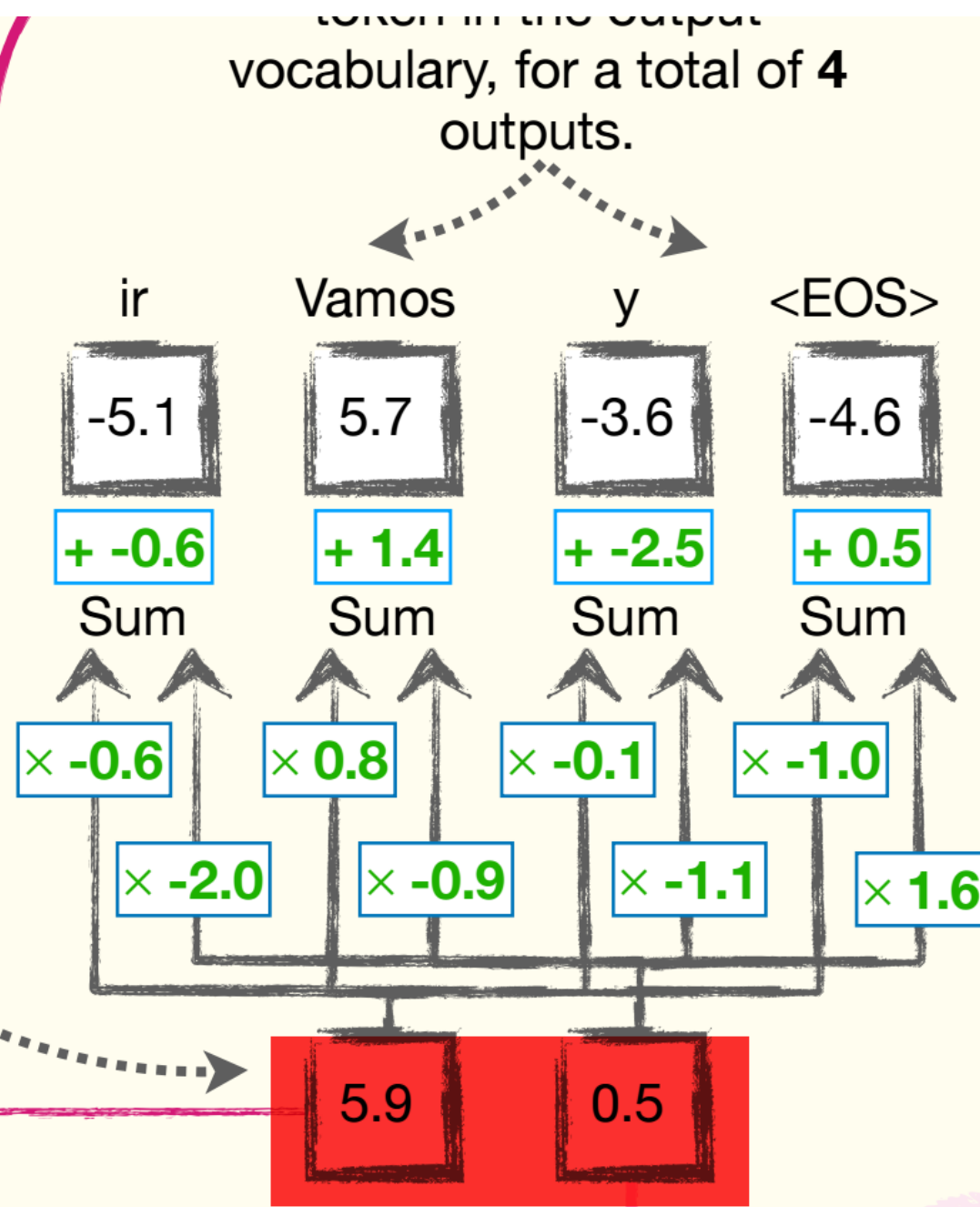
Encoder

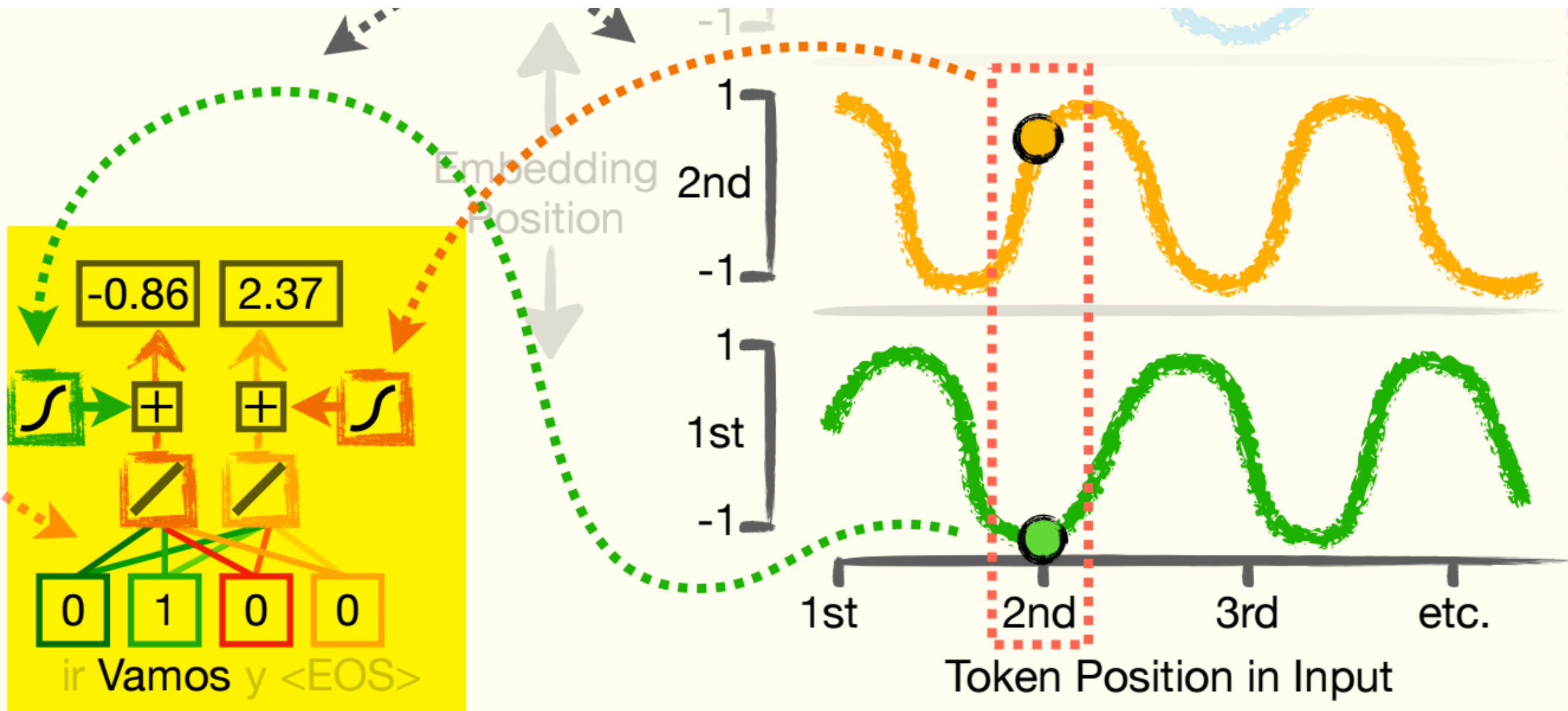


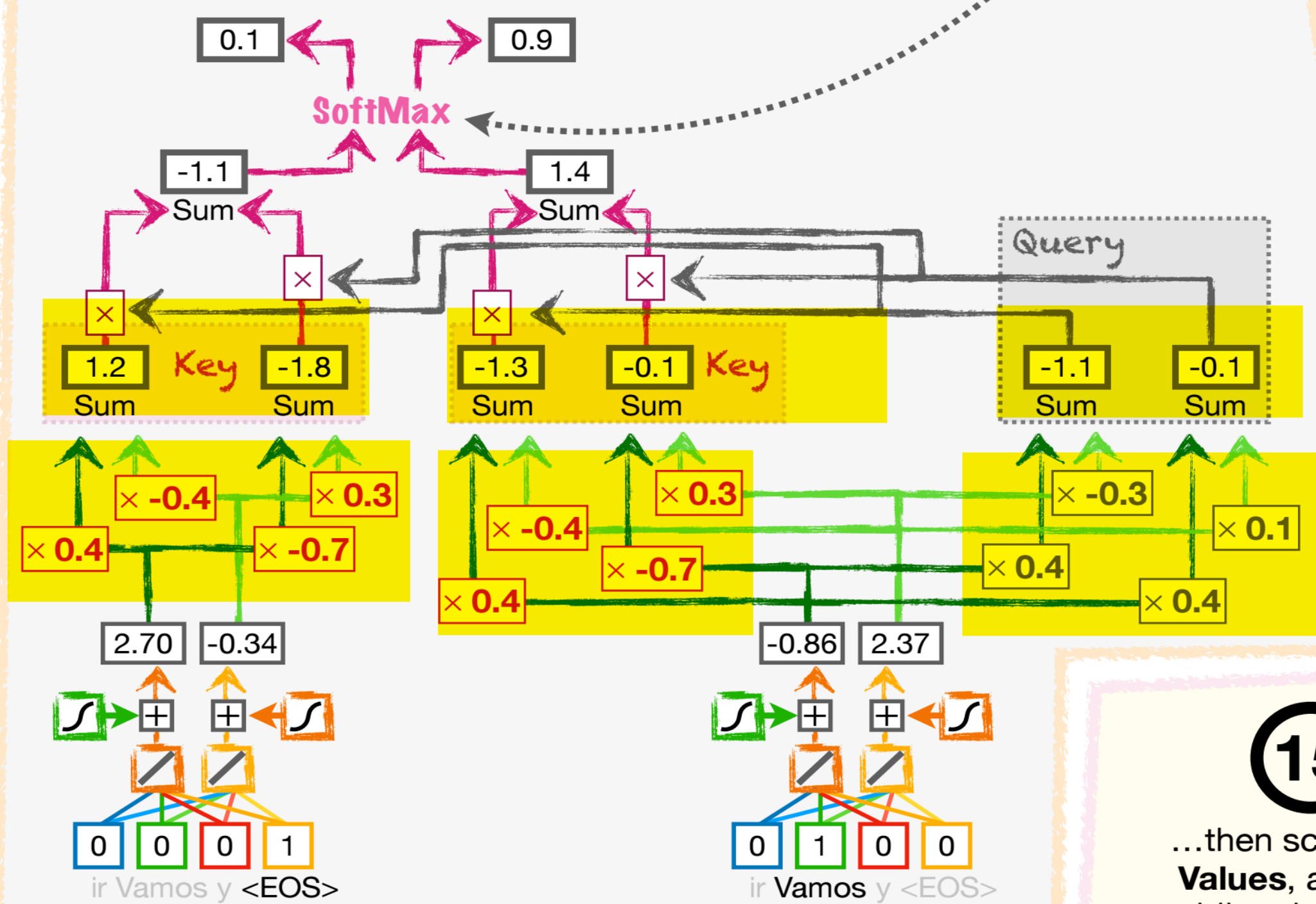
...
De
on
th
i
pre
o
P



We run the **2** values through a Fully Connected Layer that has one input for each value that represents the current token. So, in this case, we have **2** inputs...

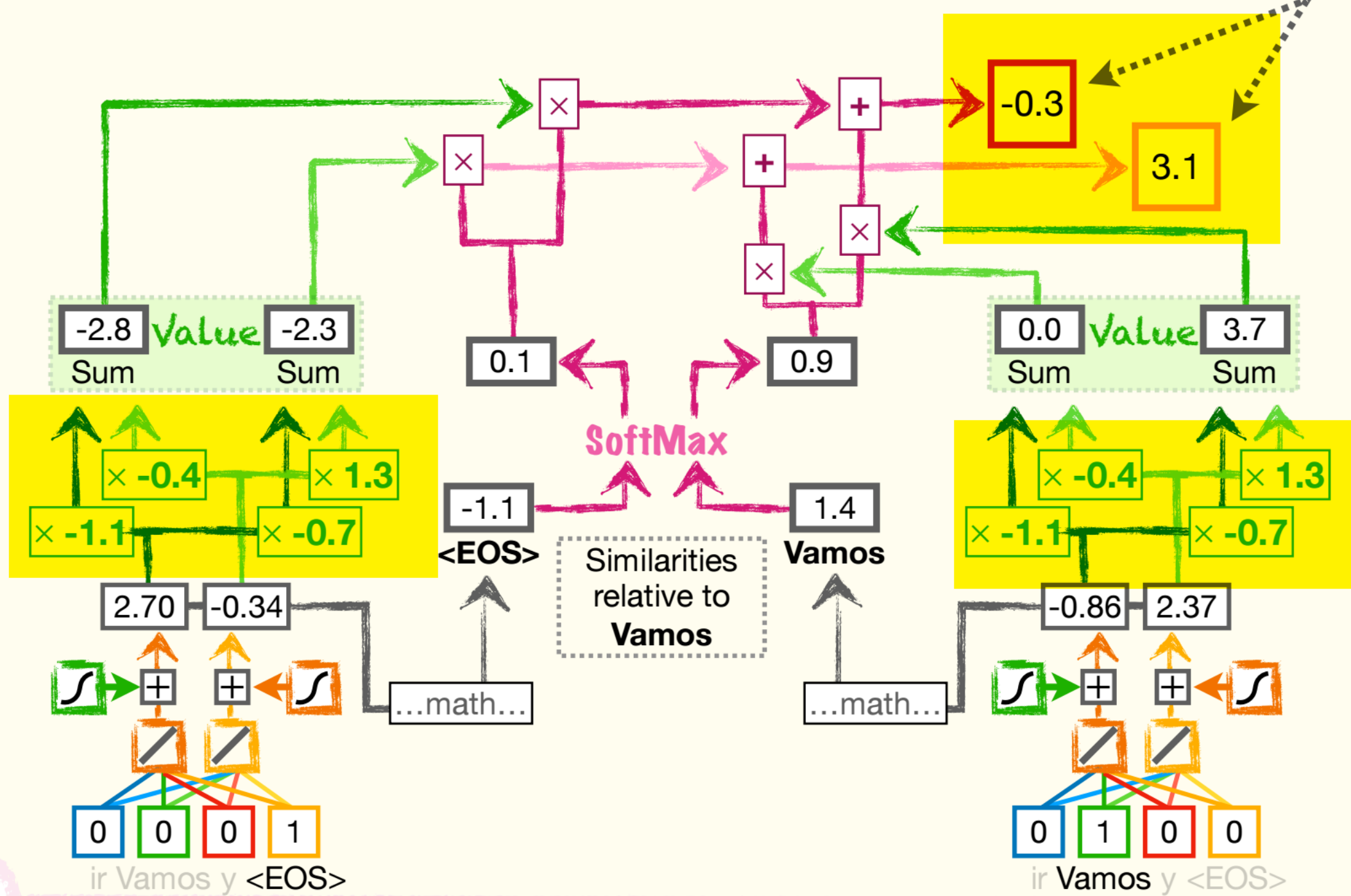


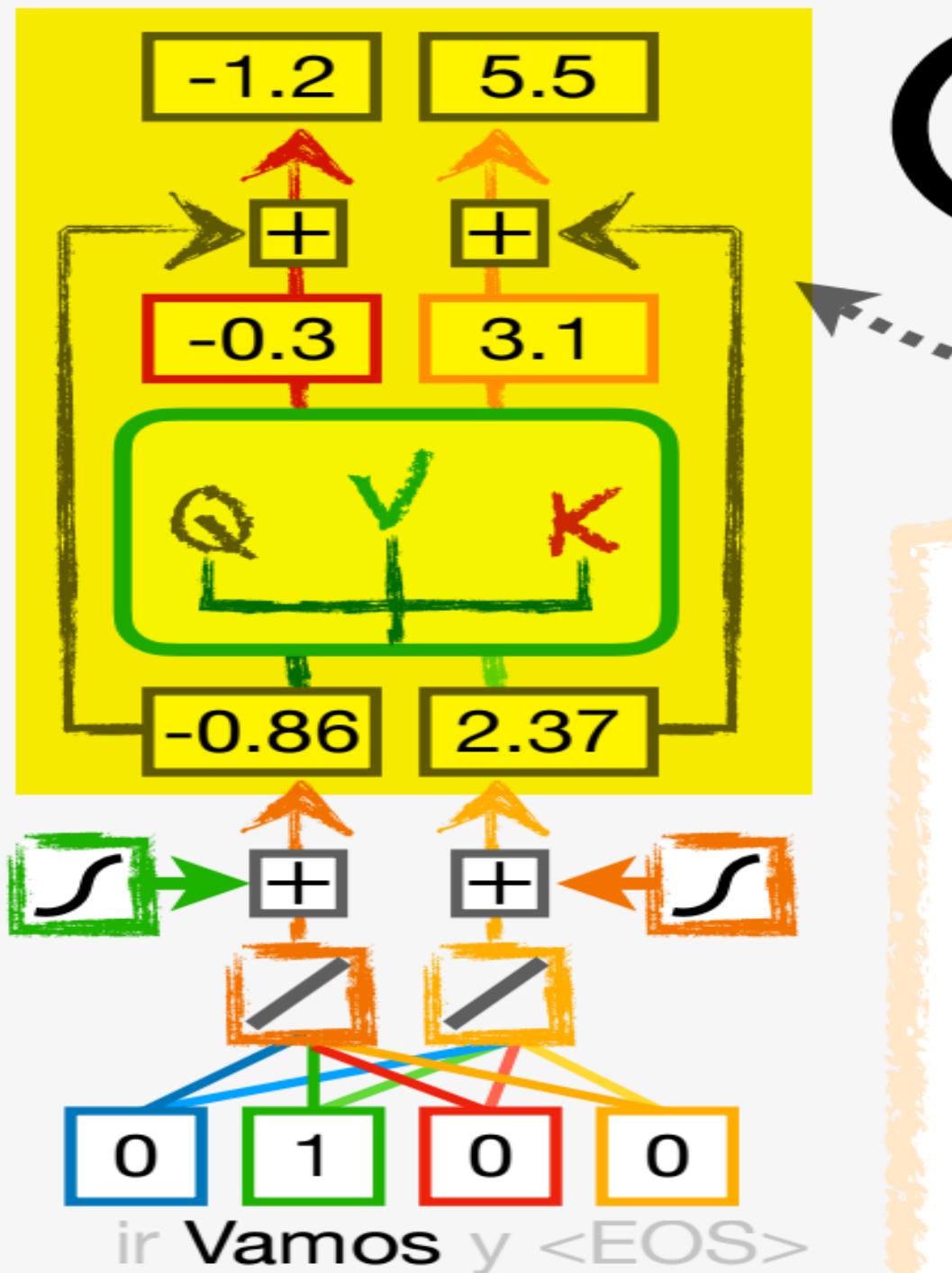


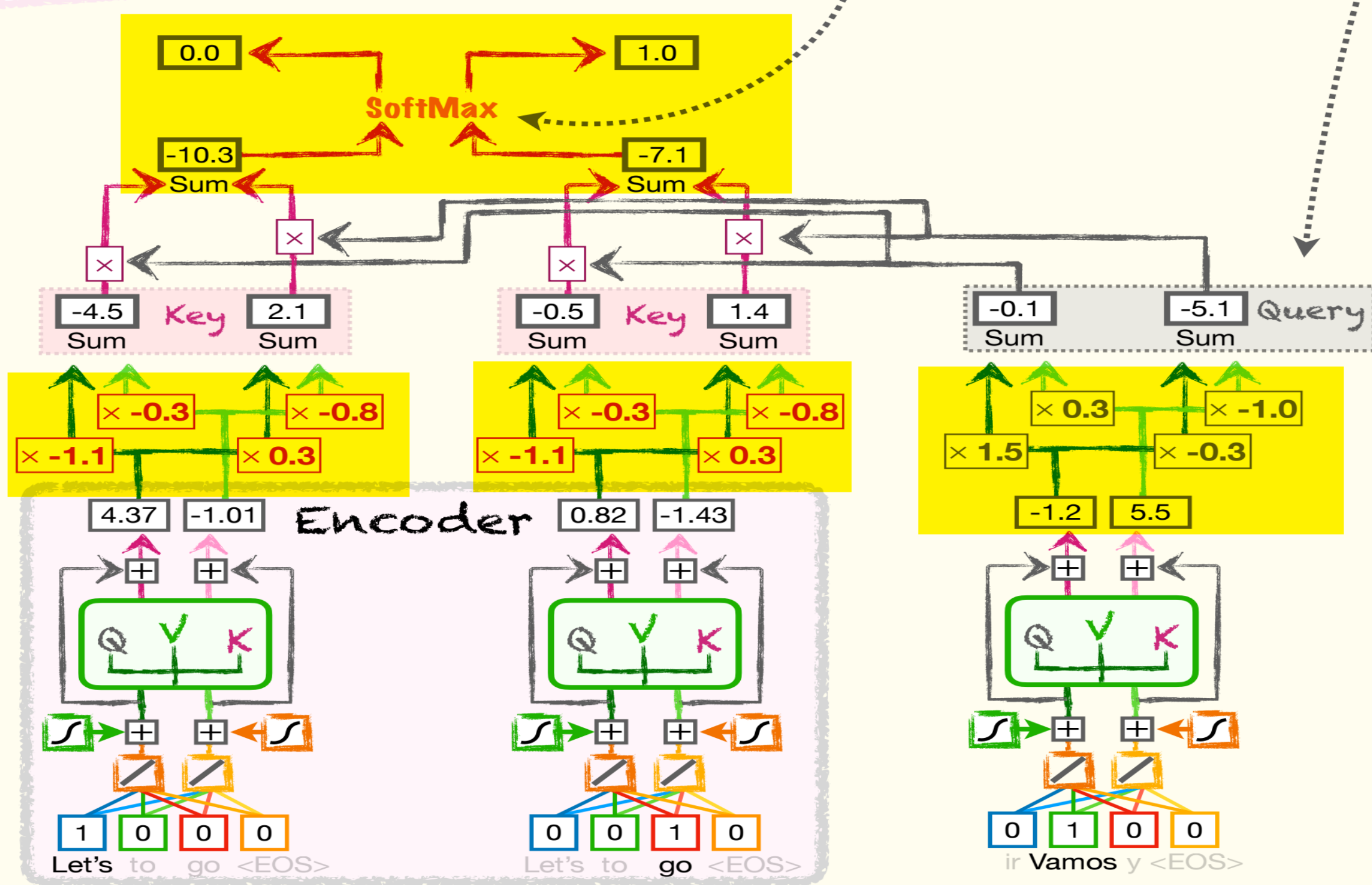


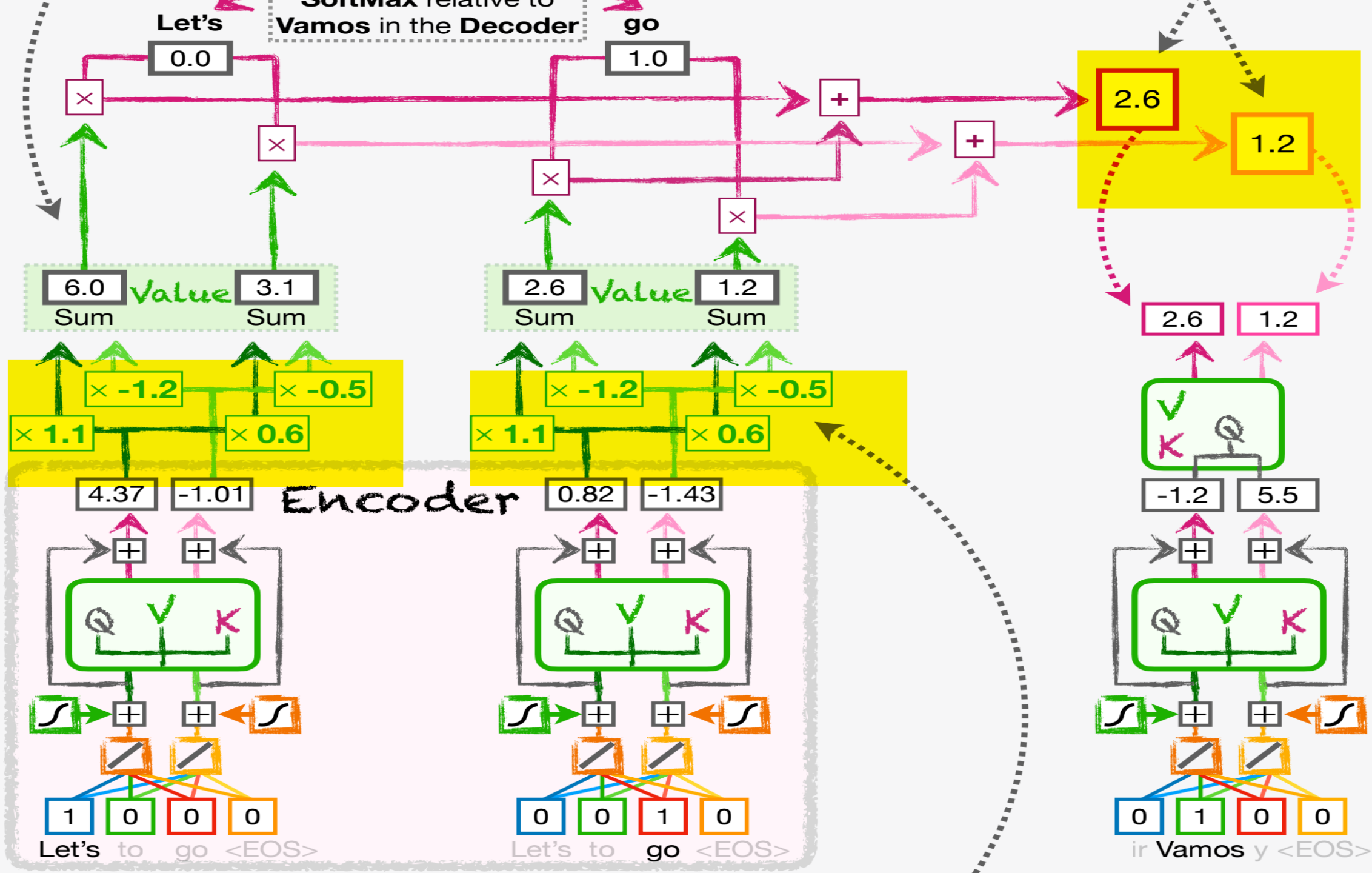
15

...then scale
Values, and
adding the p









ir Vamos y <EOS>

0	1	0	0
---	---	---	---

SoftMax

-5.1	5.7	-3.6	-4.6
------	-----	------	------

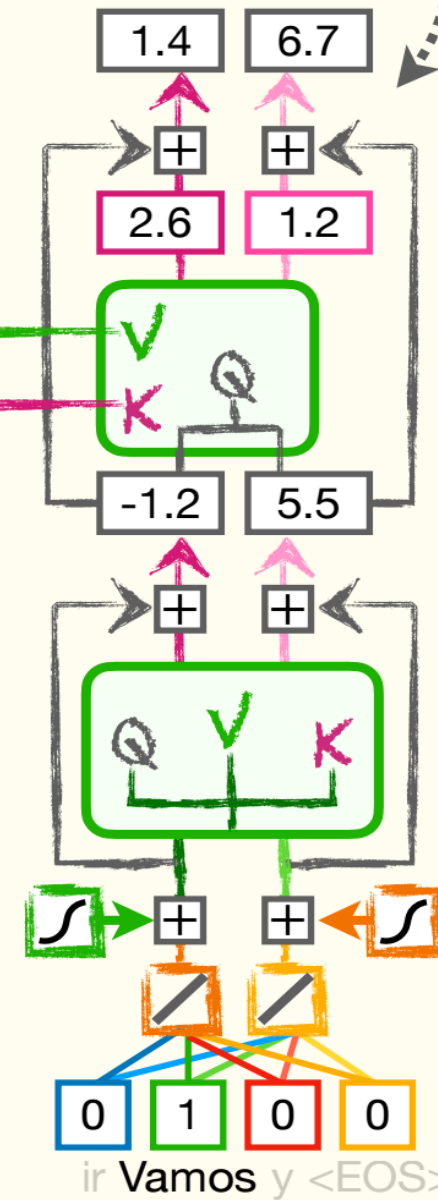
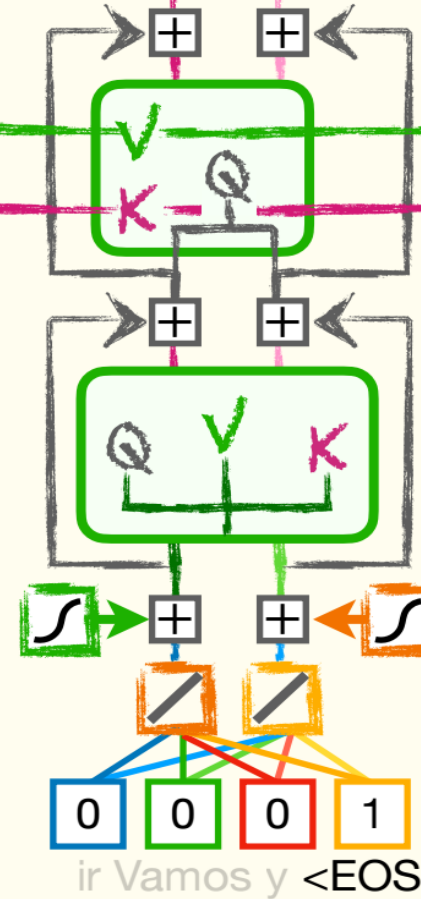
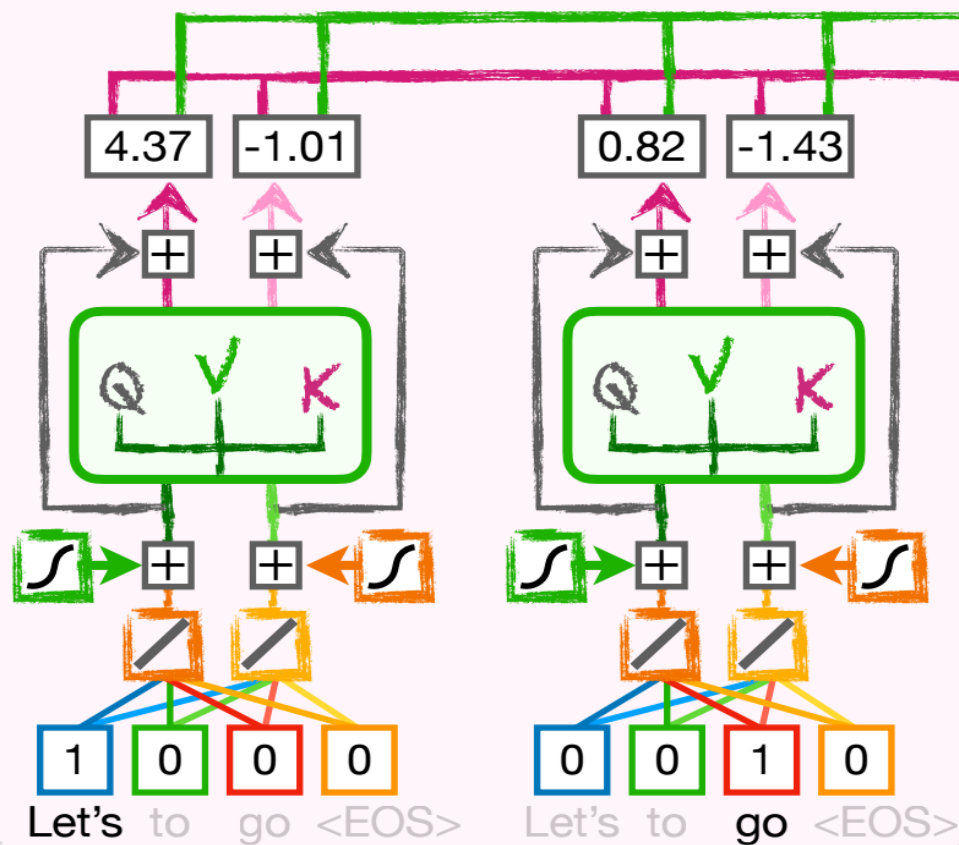
5.9	0.5
-----	-----

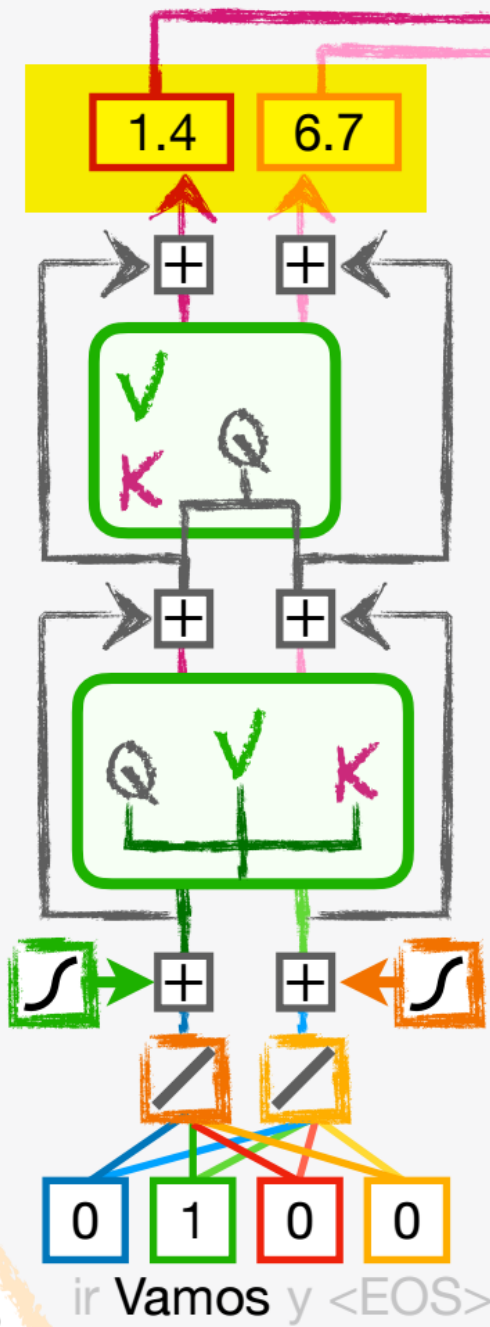
1.4	6.7
-----	-----

2.6	1.2
-----	-----

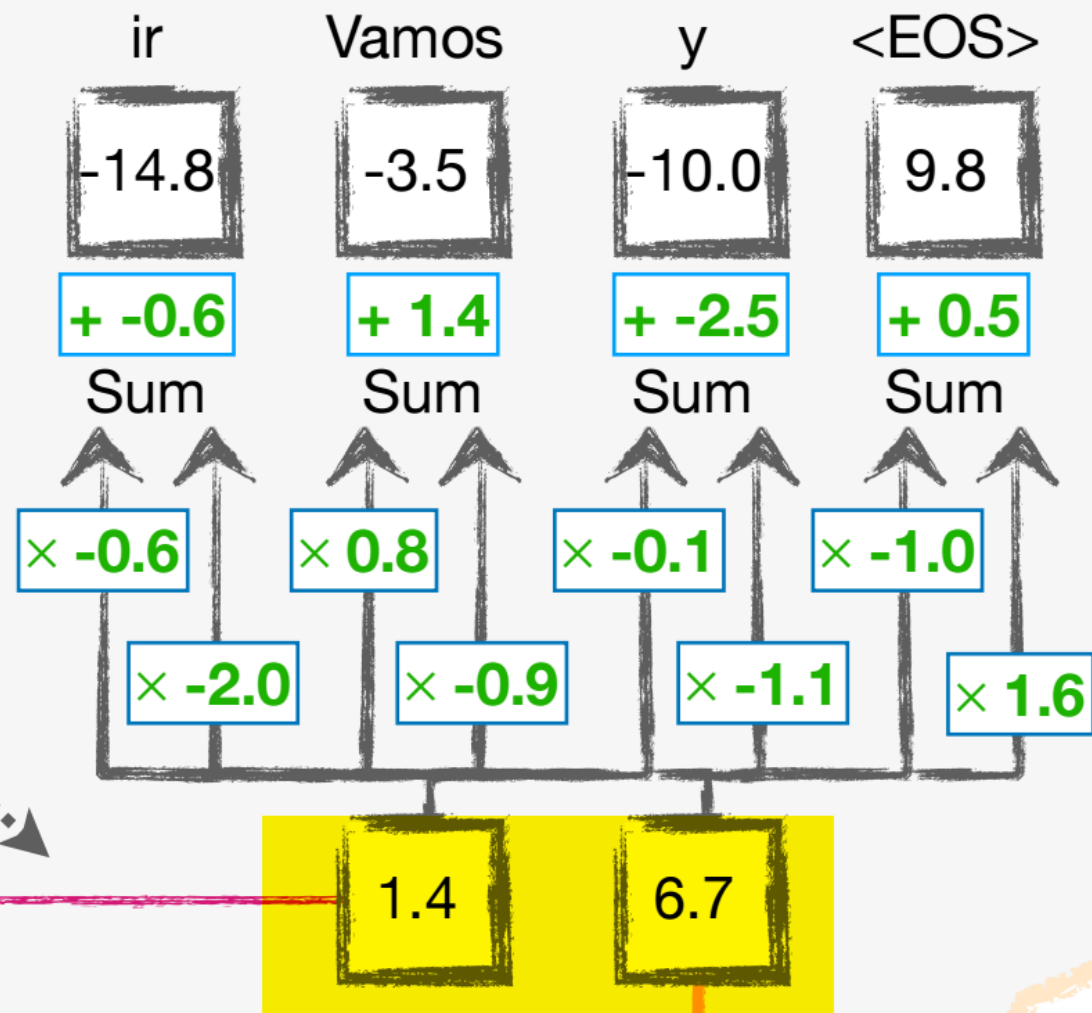
-1.2	5.5
------	-----

Encoder

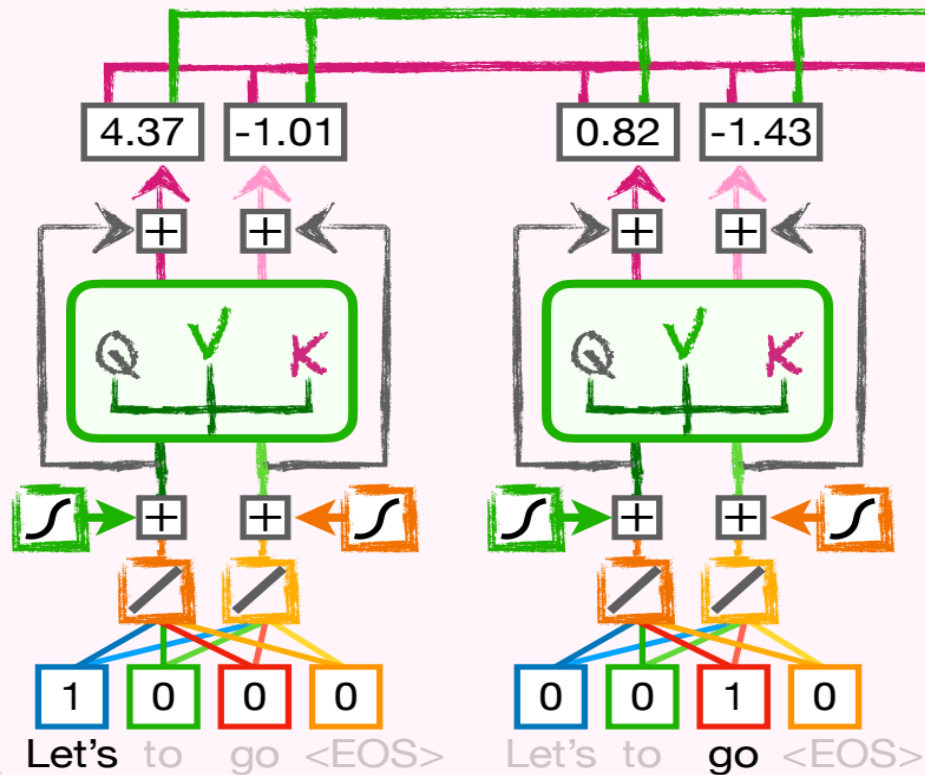




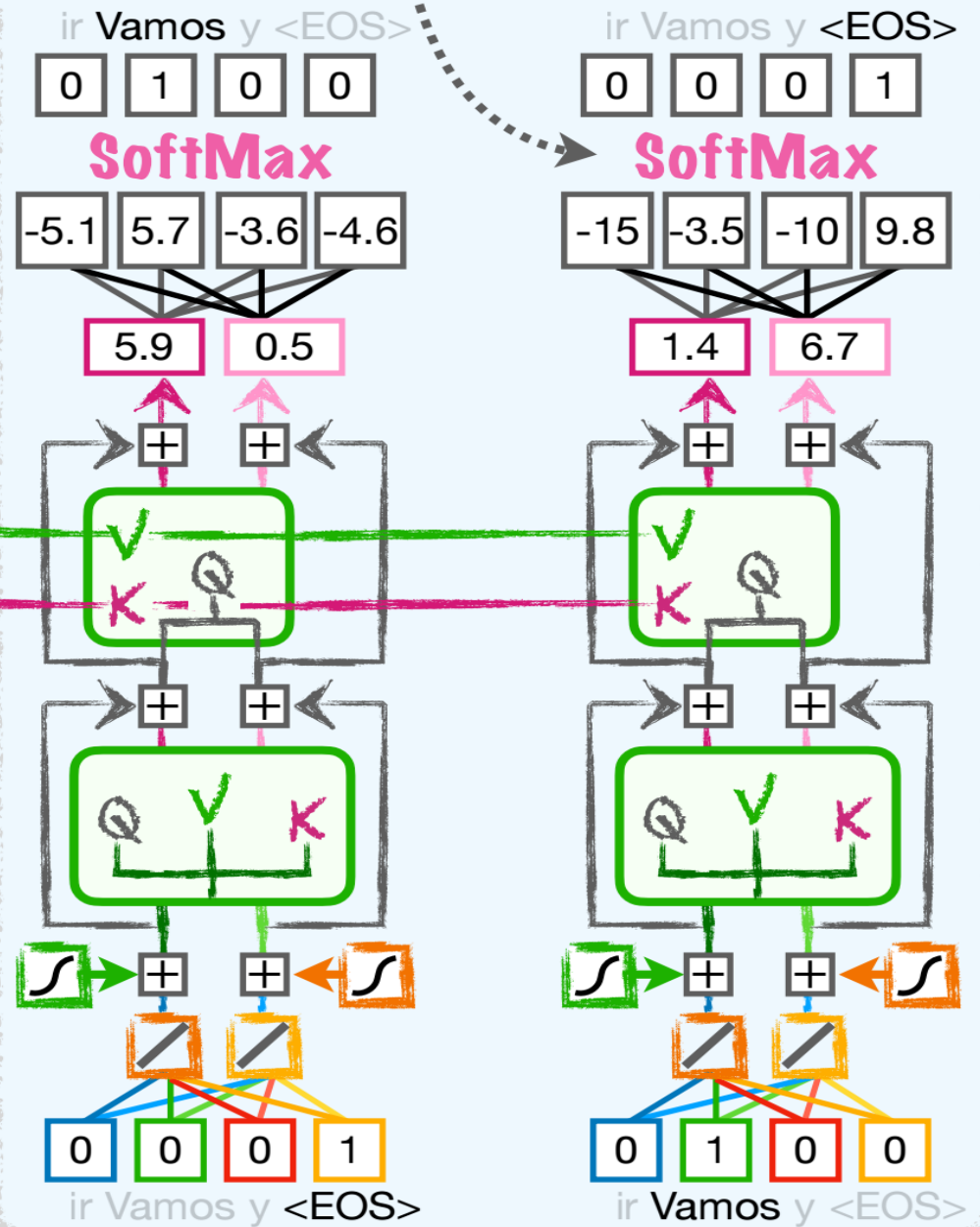
...and then run the sums
through the Fully
Connected Layer...



Encoder



Decoder



Encoder

Lets	go		
[1.87, 0.09]	[-0.78, 0.27]		
Positional encoding			
+ [0.0, 1, 0]	+ [-0.9, 0.4]		
[1.87, 1.09]	[-1.68, 0.67](1)		
Attention calculation in encoder (Lets go)			
Q=1.1 0.6 -2.8 2.4	K = -1.7 0.5 -1.5 0.9	V = 1.5 -1.0 -0.3 -0.2	
QL = Lets X Q	KL = Lets X K	VL = Lets X V	
QG = go X Q	KG = go X K	VG = go X V	
QL = [-1.0, 3.7]	KL = [-4.7, 1.9]	VL = [2.5, -2.1]	
QG = [-3.7, 0.6]	KG = [1.9, -0.2]	VG = [-2.7, 1.5]	
QLxKL = 11.7	QLxKG = -2.6	QGxKL = 18.5	QGxKG = -7.2
Softmax(11.7, -2.6)	Softmax(18.5, -7.2)		
1.0, 0.0	1.0, 0.0		
VLx1.0 + VGx0	VLx1.0 + VGx0.0		
[2.5, -2.1]	[2.5, -2.1]		

From (1) [1.87, 1.09] [-1.68, 0.67]

+ [2.5, -2.1]

+ [2.5, -2.1]

[4.37, -1.01]

[0.82, -1.43]

Decoder

Attention calculation in decoder (EOS)

EOS [2.70, -1.34] + positional encoding [0.0, 1.0] = [2.70, -0.34]

Q=0.4 0.4
-0.3 0.1

K = 0.4 -0.7
-0.4 0.3

V = -1.1 -0.7
-0.4 1.3

QE = EOSXQ KE = EOSXK VE = EOSXV

QE = [1.2, 1.0] KE = [1.2, -1.8] VE = [-2.8, -2.3]

QEXKE = -0.4 Softmax(-0.4) = 1.0

EOS[2.70, -0.34] + VE X 1.0 = [-0.1, -2.6].....(2)

$$PE(pos, 2i) = \sin \frac{pos}{10000^{\frac{2i}{d_{model}}}}$$

$$PE(pos, \frac{2i}{+1}) = \cos \frac{pos}{10000^{\frac{2i}{d_{model}}}}$$

Decoder Encoder cross attention calculation (EOS, Lets, go)

$$Q = \begin{bmatrix} 1.5 & -0.3 \\ 0.3 & -1.0 \end{bmatrix} \quad K = \begin{bmatrix} -1.1 & 0.3 \\ -0.3 & -0.8 \end{bmatrix} \quad V = \begin{bmatrix} 1.1 & 0.6 \\ -1.2 & -0.5 \end{bmatrix}$$

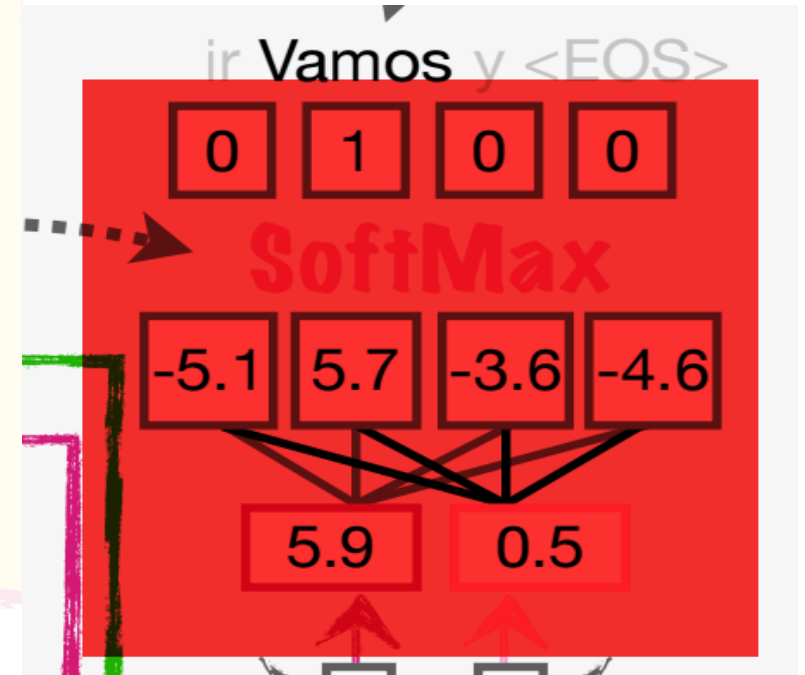
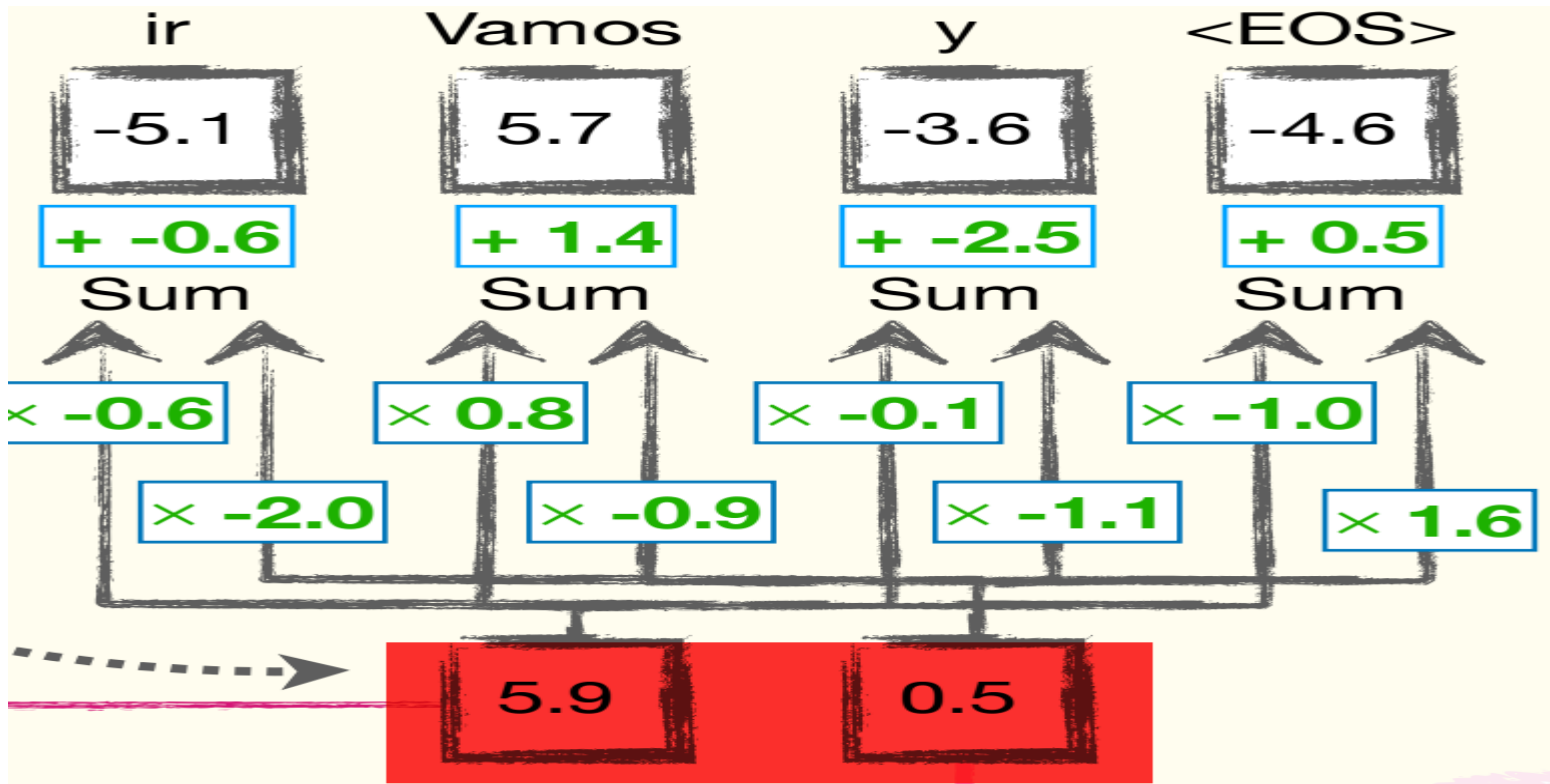
$$QE = \text{EOS} \times Q \quad KL = \text{Lets} \times K \quad KG = \text{go} \times K \\ VL = \text{Lets} \times V \quad VG = \text{go} \times V$$

$$QE = [-0.9, 2.6] \quad KL = [-4.5, 2.1] \quad VL = [6.0, 3.1] \\ KG = [-0.5, 1.4] \quad VG = [2.6, 1.2]$$

$$QExKL = 9.5 \quad QExKG = 4.1 \\ \text{Softmax}(9.5, 4.1) \\ 1.0, 0.0 \\ VL \times 1.0 + VG \times 0 \\ [6.0, 3.1]$$

$$\text{From (2) EOS } [-0.1, -2.6] + [6.0, 3.1] = [5.9, 0.5] \dots\dots\dots(3)$$

Generating first token in decoder after processing EOS



Decoder

Attention calculation in decoder (Vamos, EOS)

Vamos $[-0.86, 2.37]$ + positional encoding $[-0.9, 0.4] = [-0.86, 2.37]$

$Q = \begin{bmatrix} 0.4 & 0.4 \\ -0.3 & 0.1 \end{bmatrix}$
 $K = \begin{bmatrix} 0.4 & -0.7 \\ -0.4 & 0.3 \end{bmatrix}$
 $V = \begin{bmatrix} -1.1 & -0.7 \\ -0.4 & 1.3 \end{bmatrix}$

$KE = EOS \times K$ $VE = EOS \times V$ $QV = \text{Vamos} \times Q$ $VV = \text{vamos} \times V$
 $KE = [1.2, -1.8]$ $VE = [-2.8, -2.3]$ $QV = [-1.3, -0.1]$ $VV = [0.0, 3.7]$

$QV \times KE = -1.1$ $QV \times KV = 1.4$
 $\text{Softmax}(-1.1, -1.4) = 0.1, 0.9$
 $\text{varmos}[-0.86, 2.37] + VE \times 0.1 + VV \times 0.9 = [-1.2, 5.5] \dots\dots\dots(4)$

Decoder Encoder cross attention (Vamos, Lets, go)

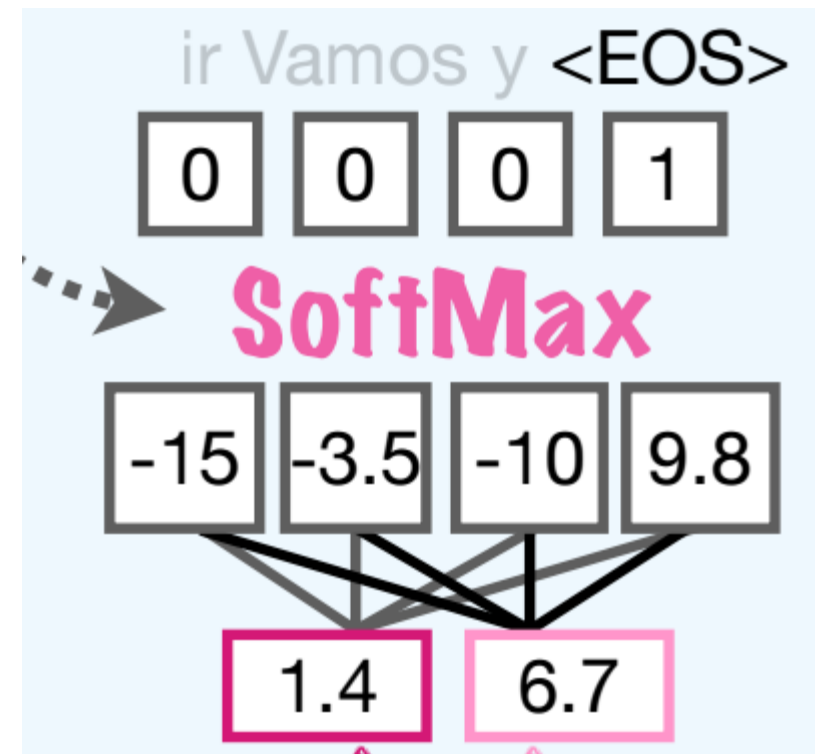
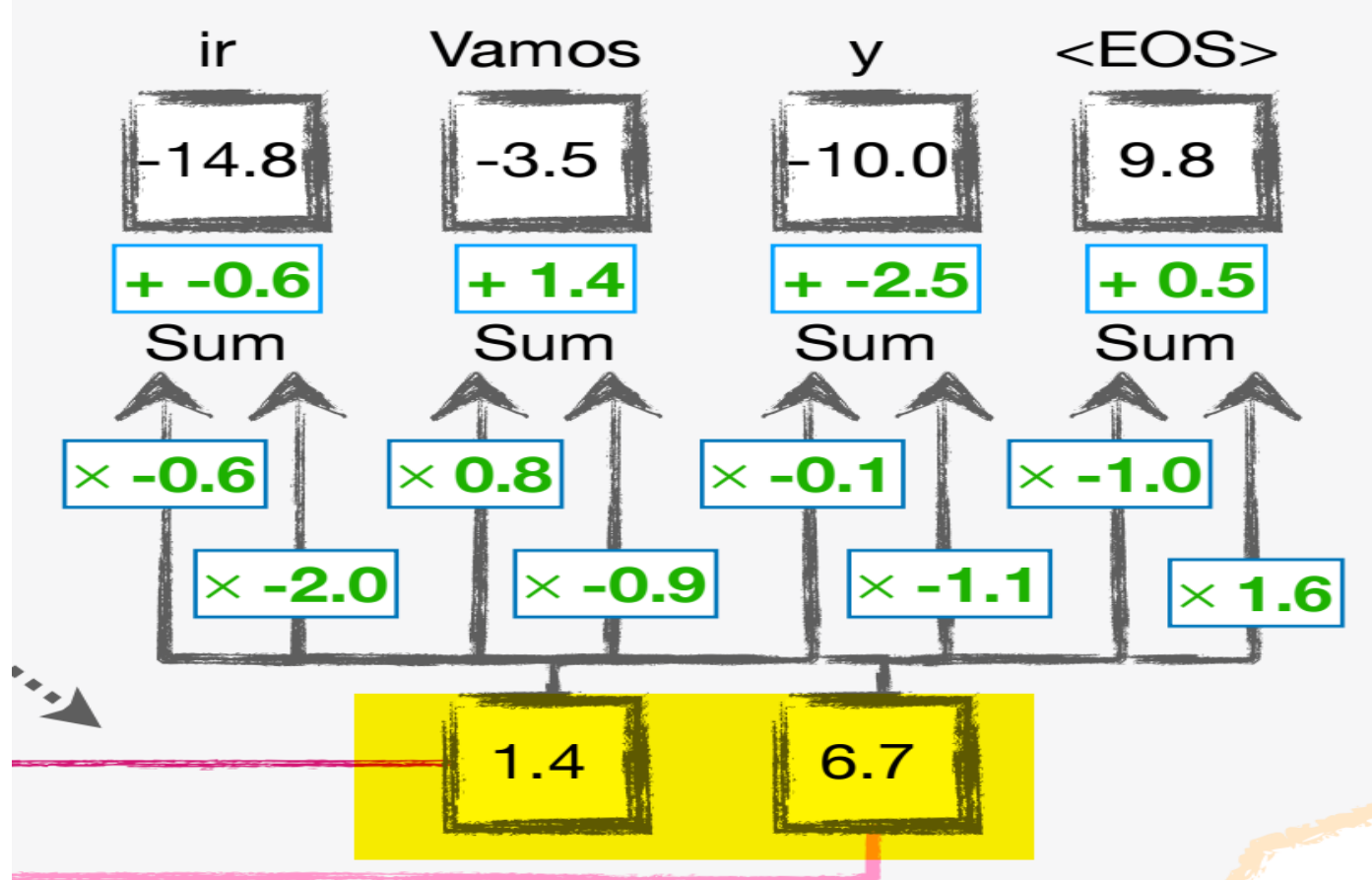
$Q = \begin{bmatrix} 1.5 & -0.3 \\ 0.3 & -1.0 \end{bmatrix}$
 $K = \begin{bmatrix} -1.1 & 0.3 \\ -0.3 & -0.8 \end{bmatrix}$
 $V = \begin{bmatrix} 1.1 & 0.6 \\ -1.2 & -0.5 \end{bmatrix}$

$QV = \text{vamos} \times Q$ $KL = \text{Lets} \times K$ $KG = \text{go} \times K$
 $VL = \text{Lets} \times V$ $VG = \text{go} \times V$

$QV = [-0.1, -5.1]$ $KL = [-4.5, 2.1]$ $VL = [6.0, 3.1]$
 $KG = [-0.5, 1.4]$ $VG = [2.6, 1.2]$

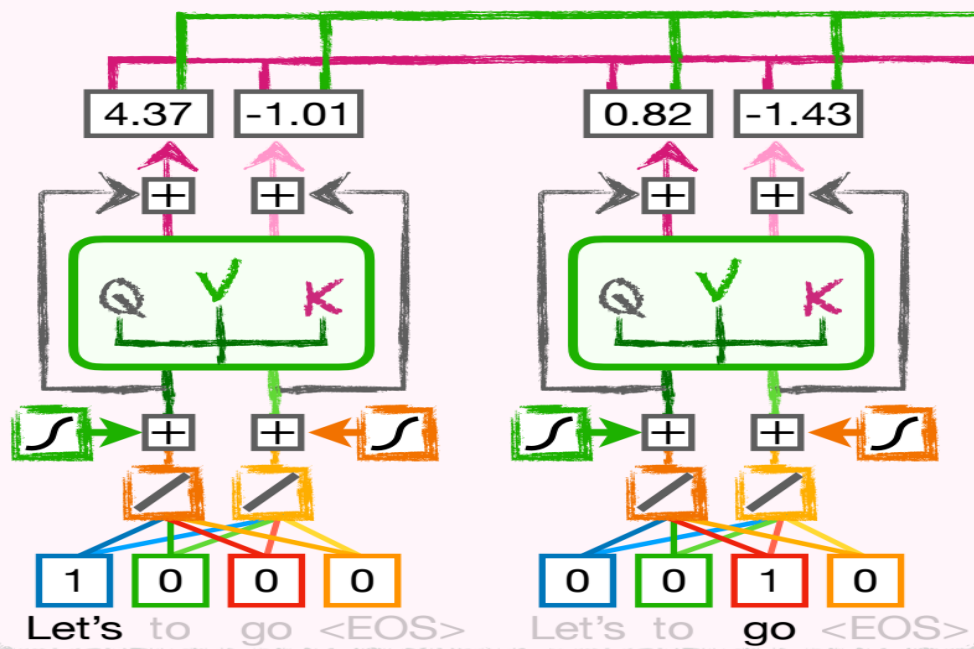
$QV \times KL = -10.3$ $QV \times KG = -7.1$
 $\text{Softmax}(-10.3, -7.1)$
 $0.0, 1.0$
 $VL \times 0.0 + VG \times 1.0$
 $[2.6, 1.2]$

From (4) Vamos $[-1.2, 5.5] + [2.6, 1.2] = [1.4, 6.7]$
(5)



and we're done decoding.

Encoder



Decoder

